

Build a real-time application using Angular, .NET Core and SignalR

Introduction

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Trimble

Today’s agenda

- About us
- About you
- Course Structure
- Angular
- .Net Core. Microservices
- SignalR
- Evaluation



About us





About you

- Do you have any web development background?
- Do you have any Angular background?
- What are your expectations?



What we want

- Good vibes and positive attitude
- Continuous feedback
- A pinch of dedication.



Course structure

UI: Angular

- HTML + CSS
- Angular project structure. Data binding. Pipes. Directives. Lifecycle hooks
- Routing. Angular Forms
- Angular services. Dependency injection. Observables
- Communication with server. HTTP requests



Course structure

Backend : .NetCore

- .NET Core Web App Project Structure
- Routing. Formatters and Content Negotiation
- CRUD operations
- Services & Dependency injection
- SignalR



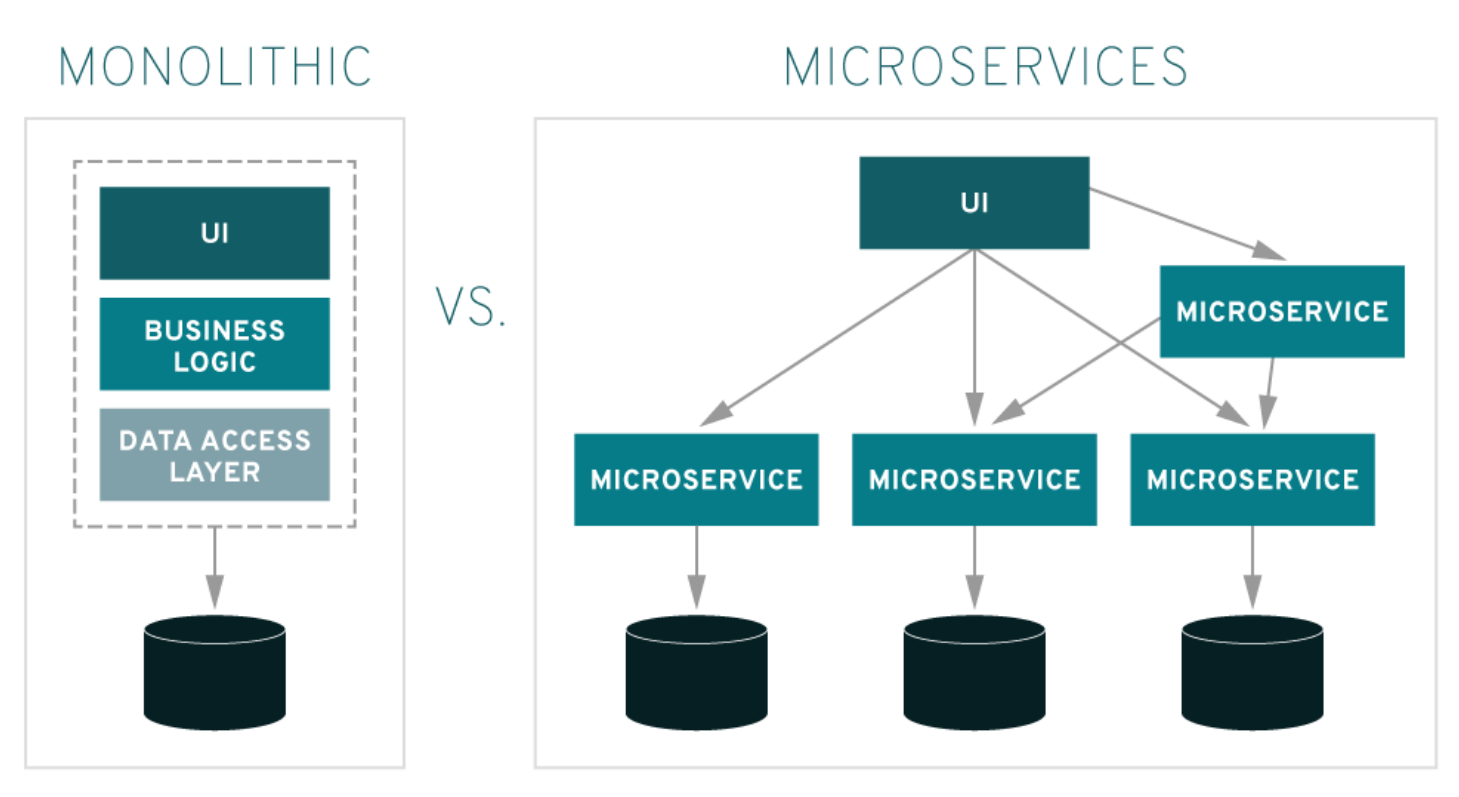
Angular


Data binding & Fastest load
Simplified MVC Pattern
Cross platform
Simple & expressive
Speed & performance
Maximum users
Flexible development
One page application

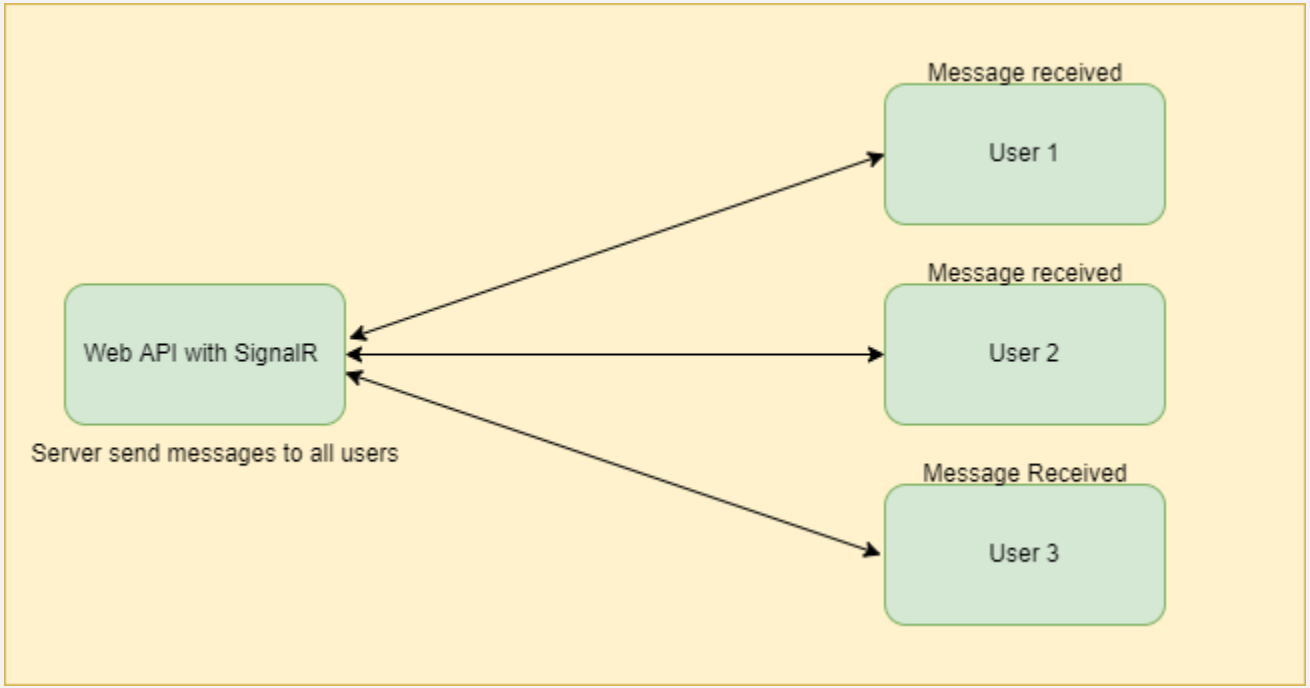
.NET Core


Complete Web Server Independence
Cross-Platform Development
Built-In Dependency Injection
Open Source and Community Focused
Side-by-Side App Versioning
Unified Stack for MVC and Web API
Rapid Development
Better Portability
Rich Controls
Better Security
More Scalable
More Maintainable

Microservices



SignalR


Message received
User 1
Message received
Web API with SignalR
User 2
Server send messages to all users
Message Received
User 3

SignalR - benefits

- Bi-directional communication between a server and one or more clients.
- Broadcasts messages to a particular client or multiple clients as per need.
- Easy way to create a real-time multi user application.



Evaluation

- Written exam: 50 %
- Oral exam: 50 %



Bye

See you next week

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information



Angular Workshop

Course 2: How a web app works. HTML & CSS

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Trimble

Agenda

- How a web app works?
- HTML
- CSS
- SCSS



How a web app works?

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

What is a web application?

- The web application is a program that runs on a browser.
- A web application addresses a particular problem in an interactive mode.



How a web application works?

- First you visit a page, for ex www.wikipedia.com.
- You introduce the web address (**URL**) in your web browser, but how does the URL look like?



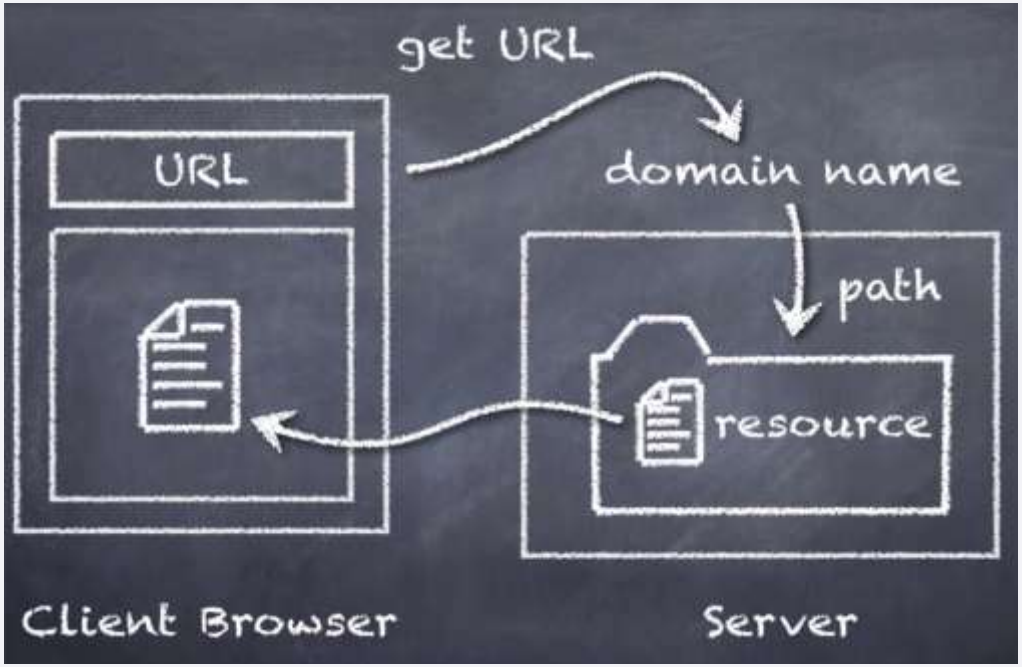
How a web application works?

- Your browser prepares to recognize this URL. It sends your request to the Domain Name Server (DNS), a repository of domain names and their IP addresses.
- The browser sends the request to the found IP address using the HTTP protocol, asking it to send a copy of the website to the client.



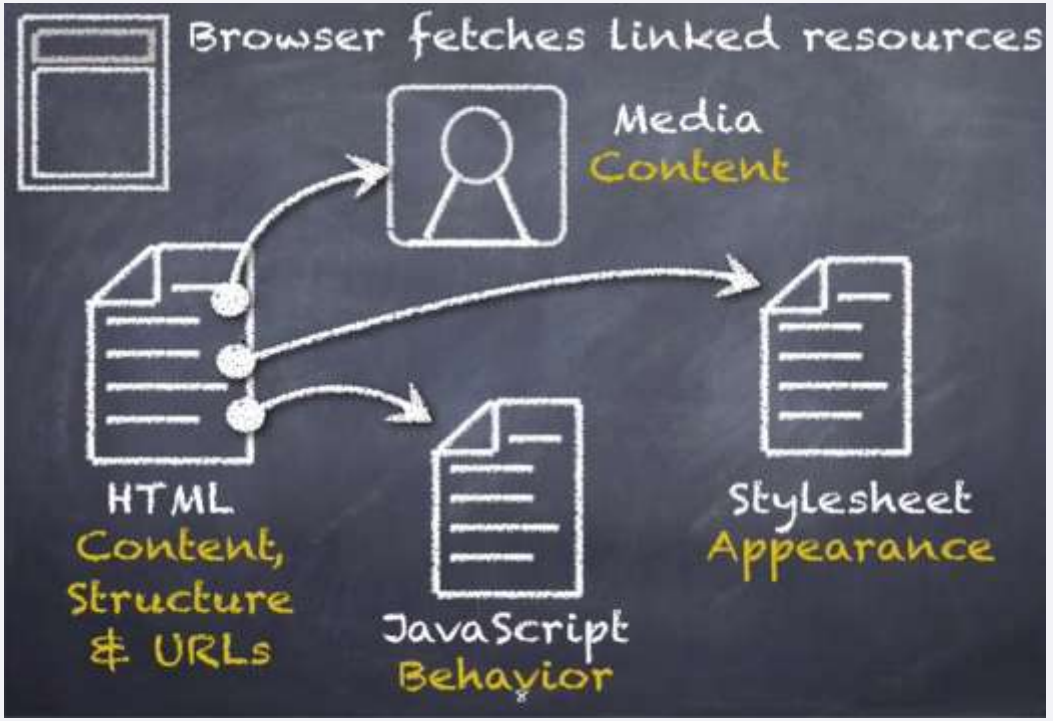
How a web application works?

- Second, the web server processes the request.
- If the server approves the client's request, the server sends the client a "200 OK" message and then starts sending the website's files to the browser as a series of small chunks called.



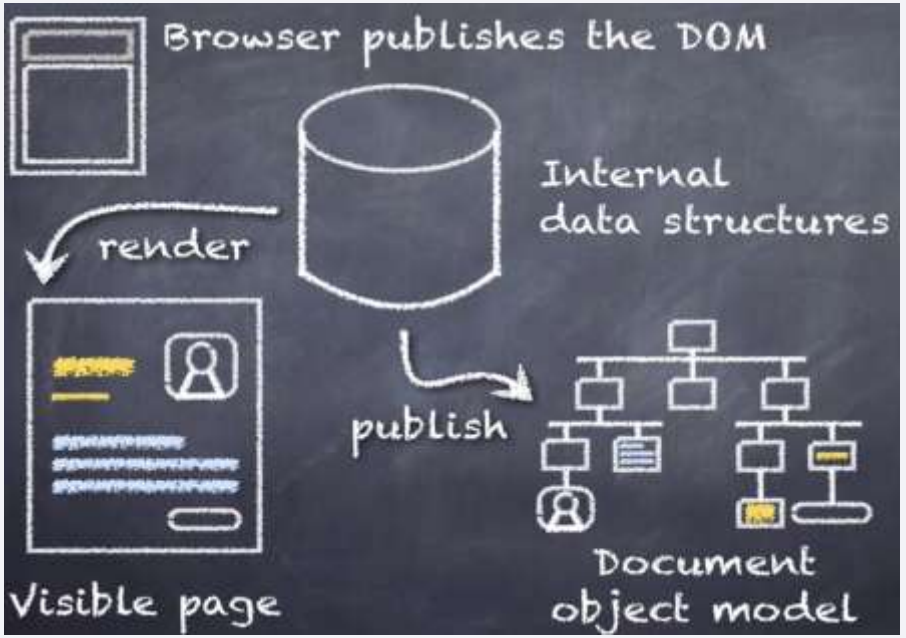
How a web application works?

- Third, the browser assembles the small chunks and parses the HTML file first that leads to the browser recognizing any <link>-element references to external CSS stylesheets and any <script>-element references to scripts.



How a web application works?

- As the browser parses the HTML, it sends requests back to the server for any CSS files and any JavaScript files, then parses the CSS and JavaScript.
- The browser generates an in-memory DOM.



- Finally, a visual representation of the page is painted to the screen, and the user sees the page content and can begin to interact with it.

Trimble

HTML

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

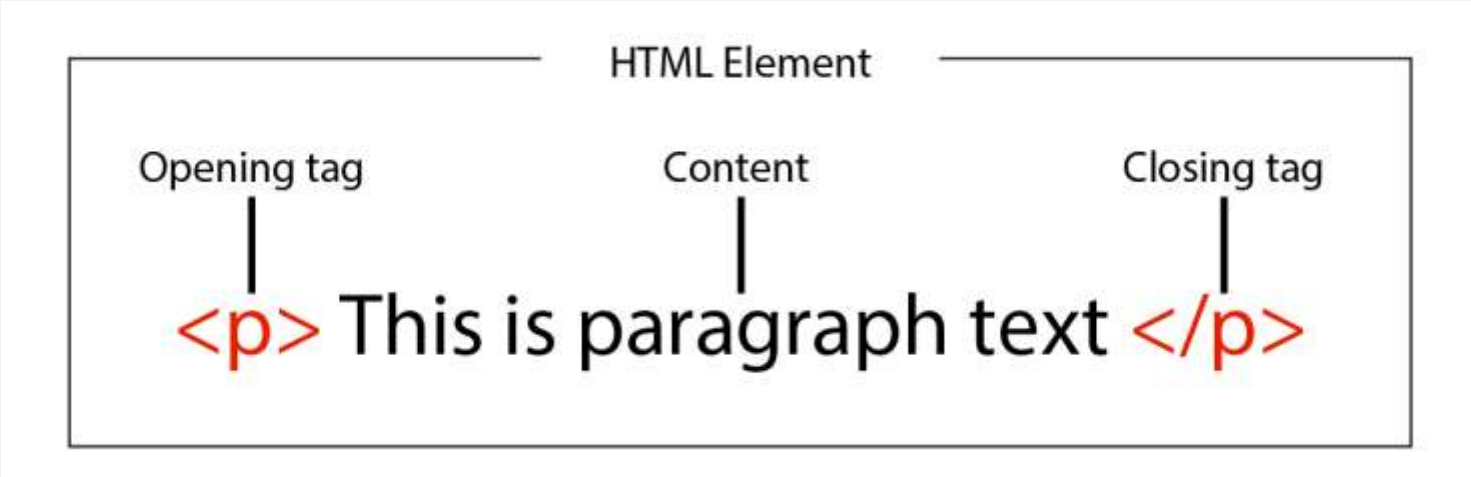
HTML

- **HTML** (HyperText Markup Language) is the most basic building block of the Web. It defines the meaning and structure of web content. Other technologies besides HTML are generally used to describe a web page's appearance/presentation (CSS) or functionality/behavior (JavaScript).
- HTML uses "markup" to annotate text, images, and other content for display in a Web browser. HTML markup includes special "elements" such as <head>, <title>, <body>, <header>, <footer>, <article>, <section>, <p>, , , <img>, <aside>, <audio>, <canvas>, <datalist>, <details>, <embed>, <nav>, <output>, <progress>, <video>, <ul>, <ol>, <li> and many others.

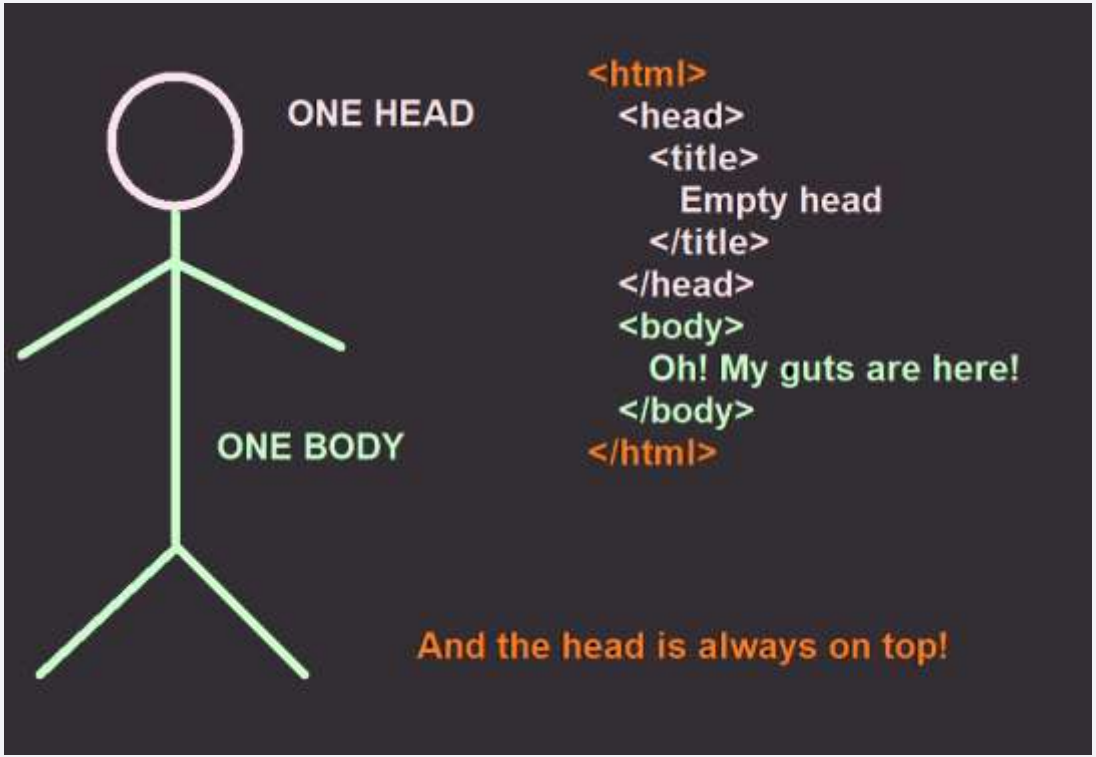


HTML

An HTML element is set off from other text in a document by "tags", which consist of the element name surrounded by "<" and ">". The name of an element inside a tag is case insensitive. That is, it can be written in uppercase, lowercase, or a mixture. For example, the `<title>` tag can be written as `<Title>`, `<TITLE>`, or in any other way.



HTML Page structure



HTML attributes

- All HTML elements can have **attributes**
- **Attributes** provide additional information about elements
- **Attributes** are always specified in the start tag
- **Attributes** usually come in name/value pairs like: name="value"

```
<img src="image.jpg">
```



HTML Classes

The HTML **class** attribute is used to specify a class for an HTML element.

Multiple HTML elements can share the same class. Multiple classes can be assigned to the same HTML element.

```
<h2 class="city">Paris</h2>
```



HTML IDs

The HTML **id** attribute is used to specify a unique id for an HTML element.

You cannot have more than one element with the same id in an HTML document.

```
<h1 id="myHeader">My Header</h1>
```



Trimble

CSS

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

CSS

- **CSS** (stands for **C**ascading **S**tyle **S**heets) describes how HTML elements are to be displayed on screen, paper, or in other media.
- CSS is used to define styles for web pages, including the design, layout and variations in display for different devices and screen sizes.
- The style definitions are normally saved in external .css files.
- With an external stylesheet file, you can change the look of an entire website by changing just one file!



Types of CSS

There are 3 types of CSS styles declaration are as follows:

1. Internal (Embedded) Styles (small NOPE) - Internal styles are placed inside the head section of a particular web page via the style tag. Internal styles are also called “Embedded” styles.
2. Inline Styles (big NOPE) - Inline styles are placed directly inside an HTML element in the code.
3. External Styles (big YES) - An external style sheet is a separate page which is then linked to the web page.



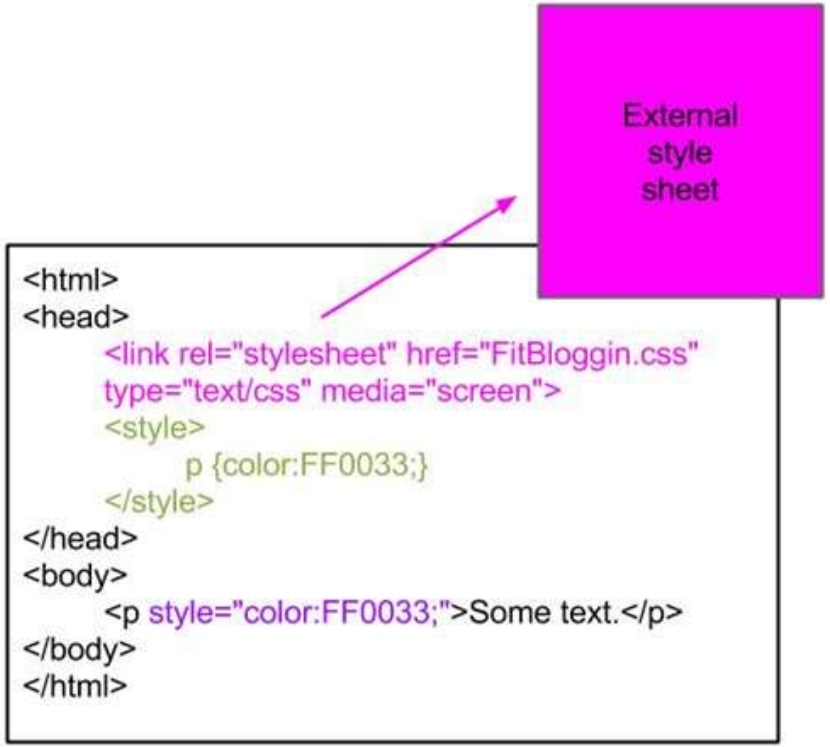
Types of CSS

CSS Linkage

How CSS is inserted:

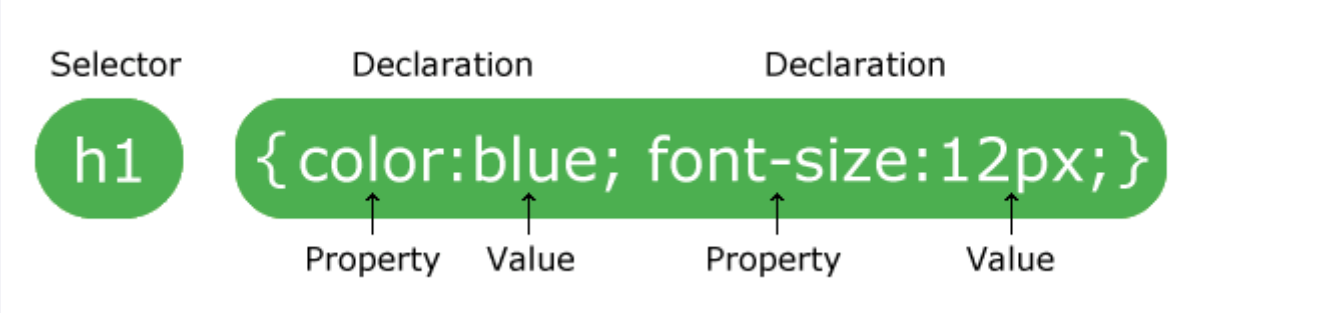
- Inline
- Internal
- External*

*most common for blogs

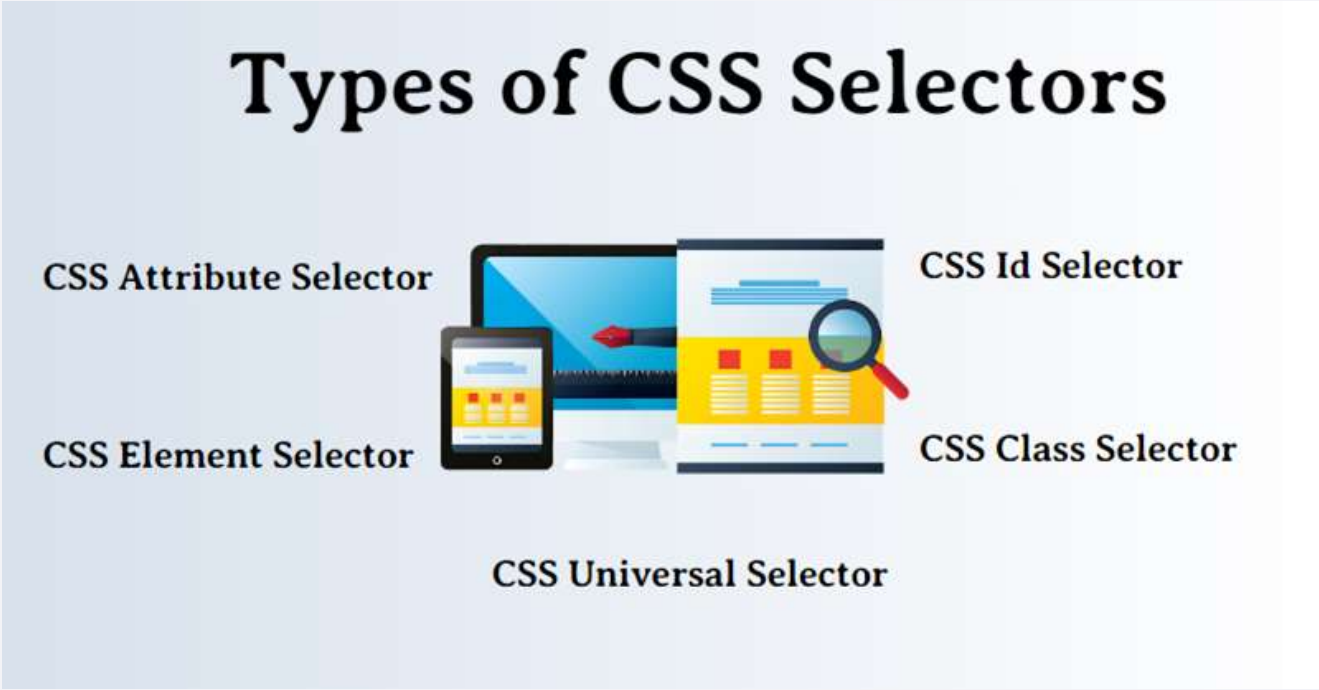


CSS Declarations

- The selector points to the HTML element you want to style.
- The declaration block contains one or more declarations separated by semicolons.
- Each declaration includes a CSS property name and a value, separated by a colon.
- Multiple CSS declarations are separated with semicolons, and declaration blocks are surrounded by curly braces.



CSS Selectors



CSS Selectors

CSS Specificity Cheat Sheet

Not Specific Enough	0.0.0.0.0	*
↕	0.0.0.0.1	Element & Pseudo-Element
Sweet Spot	0.0.0.1.0	Class, Pseudo-Class, Attribute
↕	0.0.1.0.0	ID
	0.1.0.0.0	Inline-Style
Too Specific	1.0.0.0.0	!important



CSS Combinators

Selector	Example	Example description
element element	div p	Selects all <p> elements inside elements
element>element	div > p	Selects all <p> elements where the parent is a element
element+element	div + p	Selects all <p> elements that are placed immediately after elements
element1~element2	p ~ ul	Selects every element that are preceded by a <p> element



CSS Pseudo-classes

A **pseudo-class** is used to define a special state of an element.

For example, it can be used to:

- Style an element when a user mouses over it
- Style visited and unvisited links differently
- Style an element when it gets focus

https://www.w3schools.com/css/css_pseudo_classes.asp



SCSS

- SCSS is a preprocessor which lets you use features that aren't a part of the wider CSS standard yet, and provides better workflows for maintaining your stylesheets. It is one of the syntaxes used by Sass.
- Sass is a stylesheet language that's compiled to CSS. It allows you to use variables, nested rules, mixins, functions, and more, all with a fully CSS-compatible syntax. Sass helps keep large stylesheets well-organized and makes it easy to share design within and across projects.
- More details:https://sass-lang.com/guide



Bye

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information



Angular Workshop

Course 3: Modules, Components, Data Binding, Directives, Pipes

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Trimble

Agenda

- Angular project structure
- Decorators in Angular
- Modules
- Components
- Data Binding
- @Input and @Output
- Pipes
- Directives

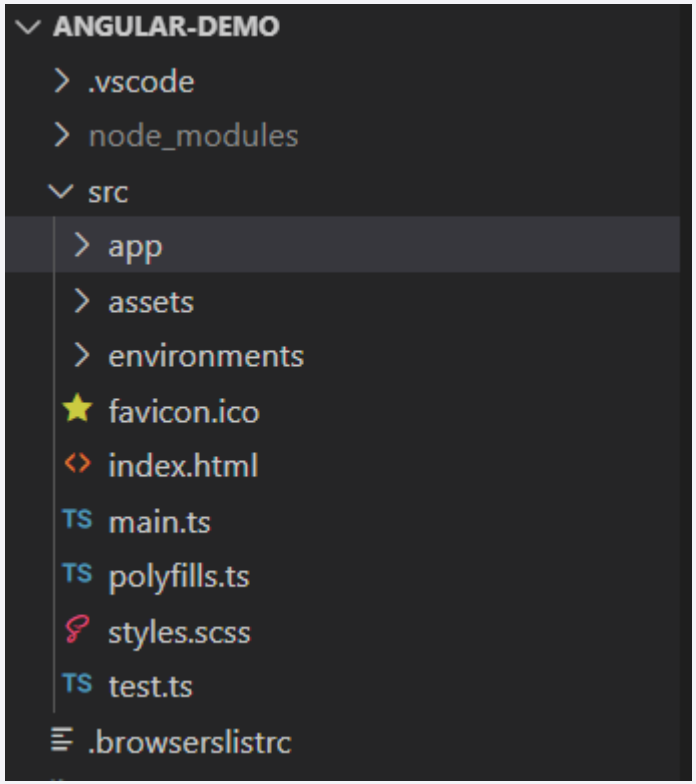


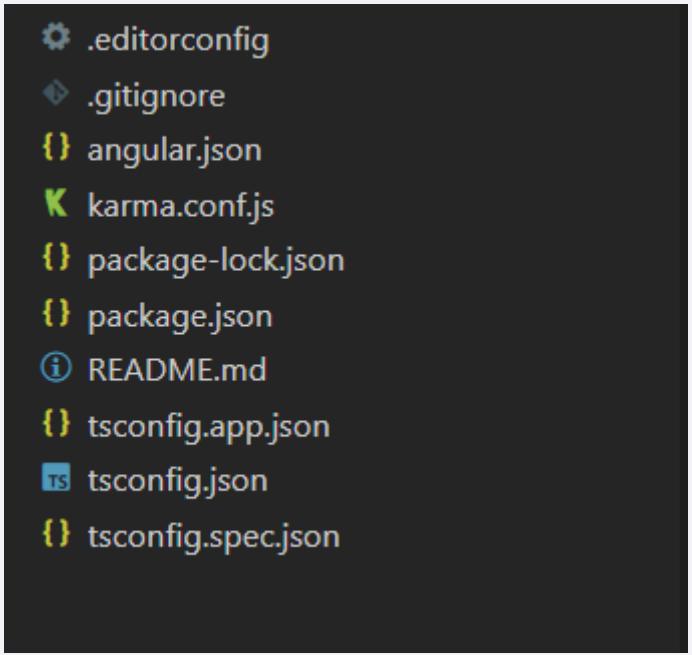
Trimble

Angular project structure

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Angular project structure





Angular project structure

- **app** folder**:** contains "modules" and "components" for our Angular application.
- **node_modules** folder: contains all packages installed for this project
- **assets** folder: contains resources such as images, styles, icons, etc.
- **environments** folder: contains the environment configuration constants that help while building the angular application.
- **favicon.io** :the icon appears in the browser tab of our application.
- **index.html** :Basic HTML file
- **main.ts**: the starting point of our application. It bootstraps/starts the AppModule from the app.module.ts file.
- **polyfills.ts**: This file is used to compile our TypeScript to specific JavaScript methods.
- **styles.css**: Global CSS file.



Angular project structure

- **tests.ts:** It is the main test file.
- **.browserslistrc file:** Browser compatibility and versions are mentioned in this file.
- **.editorconfig**: This file deals with consistency in code editors to organize some basics such as indentation and whitespaces.
- **angular.json**: This file defines the structure of our application. It includes settings associated with our application(+ env).
- **karma.conf.js**: This is the configuration file for the Karma Test Runner. It is used in Unit Testing.
- **package.json**: This is the npm configuration file. All the dependencies mentioned in this file. We can modify dependency versions as per our need on this file.
- **package-lock.json**: Whenever we change something on the node_modules or package.json, this file will be generated.



Angular project structure

- **README.md**: This file is created by default. It contains our project description.
- **tsconfig.app.json:** This configuration file overrides the tsconfig.json file with relevant app-specific configurations.
- **tsconfig.base.json**: This file has been introduced in Angular 10+. It has the same configuration as compared to tsconfig.json file.
- **tsconfig.json**:TypeScript compiler configuration file. This is responsible for compiling TypeScript to JavaScript so that the browser will understand.
- **tsconfig.spec.json**: This file overrides the tsconfig.json file with app-specific unit test configurations while running the "ng test" command.
- **tslint.json:** is a static analysis tool. This file keeps track of the TypeScript code for readability, maintainability, and functionality errors



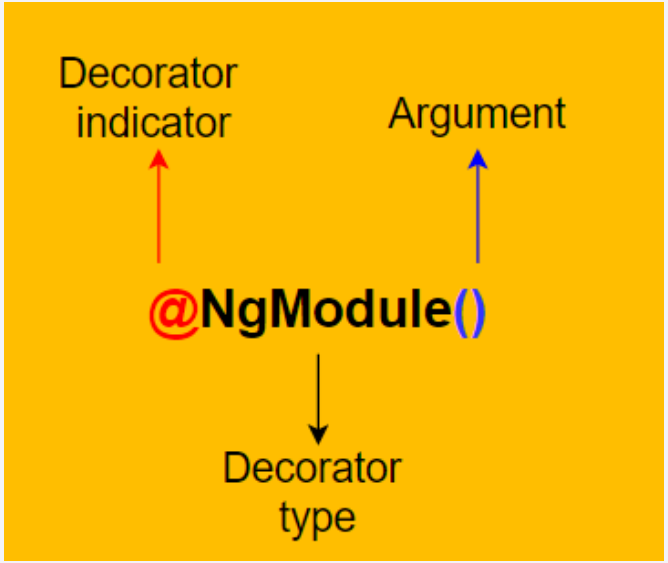
Trimble

Decorators

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Decorator

- In Angular the decorators are used to store metadata about a class, method, property or parameter.
- Each decorator has a list of arguments which can be set.
- Example:
  - Class decorator: @NgModule, @Component, @Directive;
  - Parameter decorator: @Input, @Output;



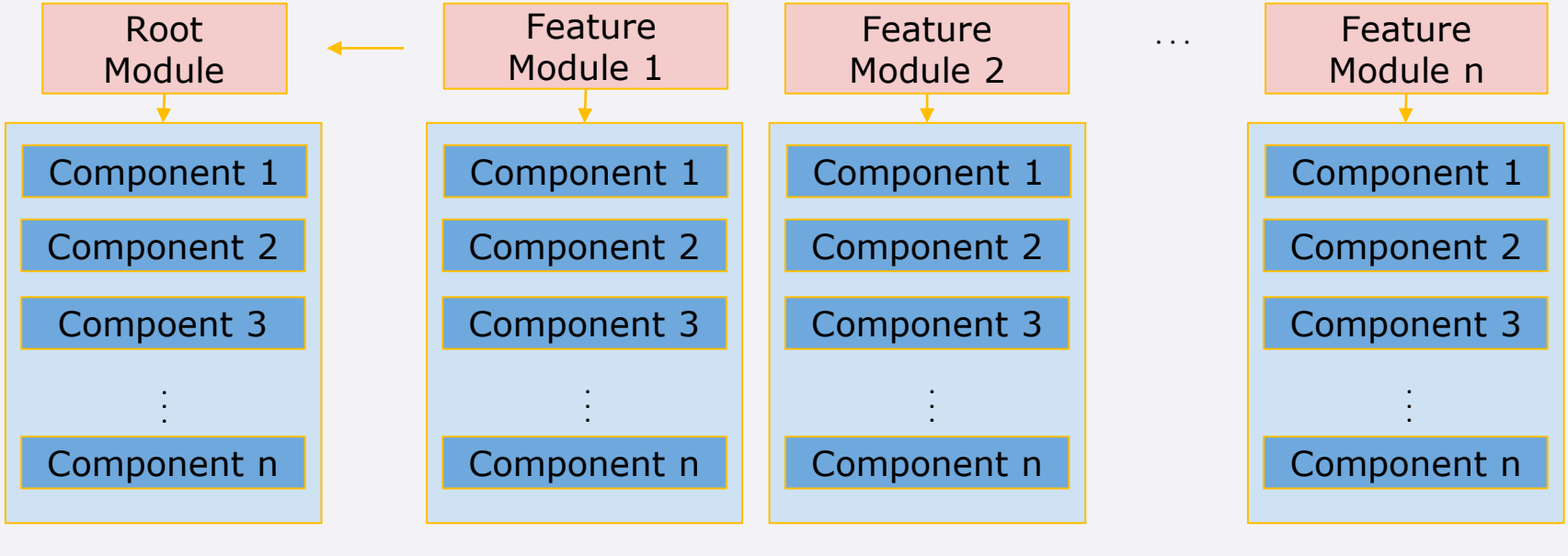
Trimble

Angular modules

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

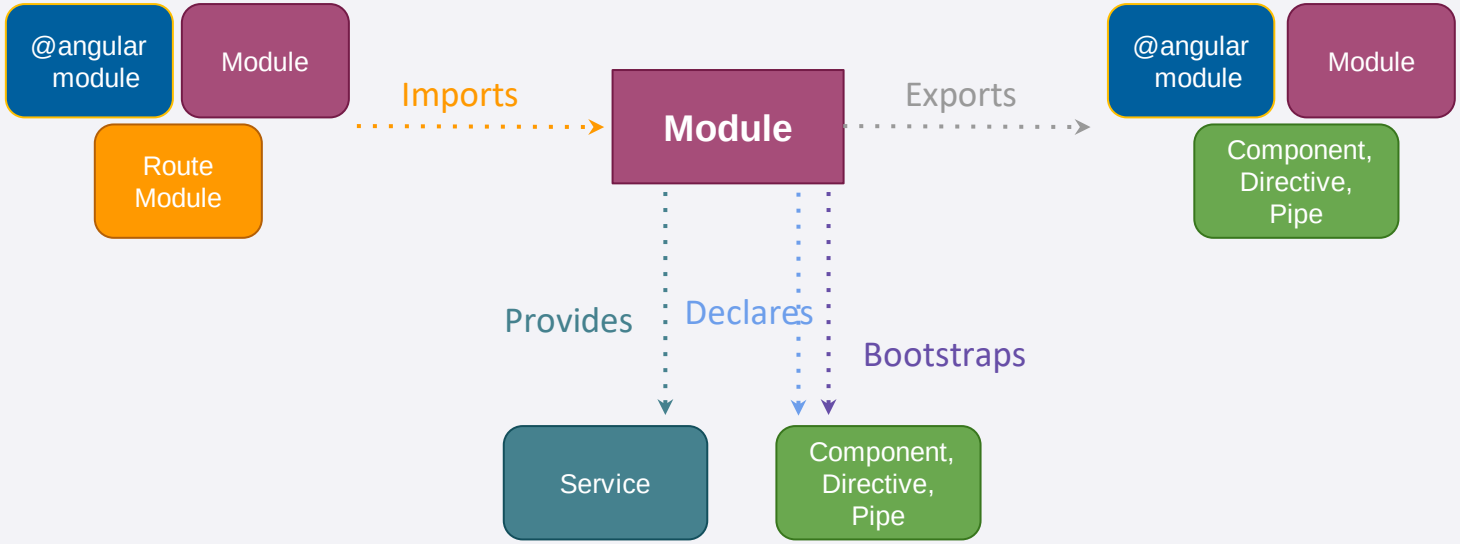
Angular modules

Angular modules keep application code organized by blocks of functionality and features. A root module acts as the starting point for an Angular application.



Angular modules

Each Angular app has a module (called **root**) to which we can link other modules called **feature modules.**



Angular modules

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
@NgModule({
  imports:      [ BrowserModule ],
  providers:    [ Logger ],
  declarations: [ AppComponent ],
  exports:      [ AppComponent ],
  bootstrap:    [ AppComponent ]
})
export class AppModule { }
```

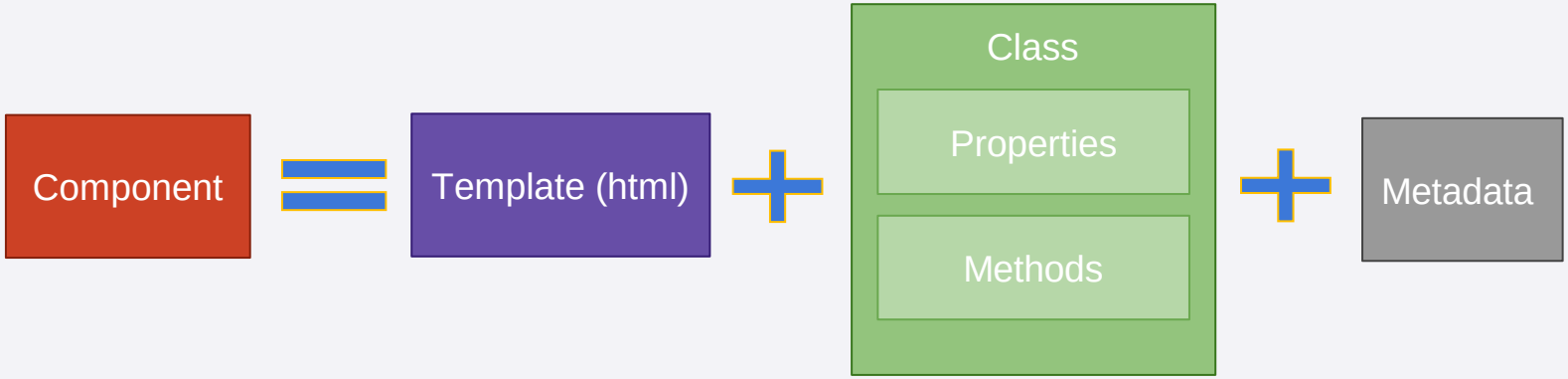


Trimble

Angular components

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Angular components



Angular components

The components are the main building blocks for a Angular application. Each component is composed of:

- HTML template: what renders on the page
- typescript class: that defines the behavior
- CSS selector : define the tag of component, this will be used to call the component in a template
- CSS styles



@Component decorator

- **selector:** a custom html tag that tells Angular to create and insert an instance of this component wherever it finds the corresponding tag in template HTML.
- **template or templateUrl:** the html code or the module-relative address of this component's HTML template.
- **styleUrls:** one or more relative paths or absolute URLs for files containing css used.
- **styles:** css used in the component

```
@Component({
  selector: 'app-note',
  templateUrl: './note.component.html',
  styleUrls: ['./note.component.scss']
})
export class NoteComponent implements OnInit {

  constructor() { }

  ngOnInit() {
  }

}
```

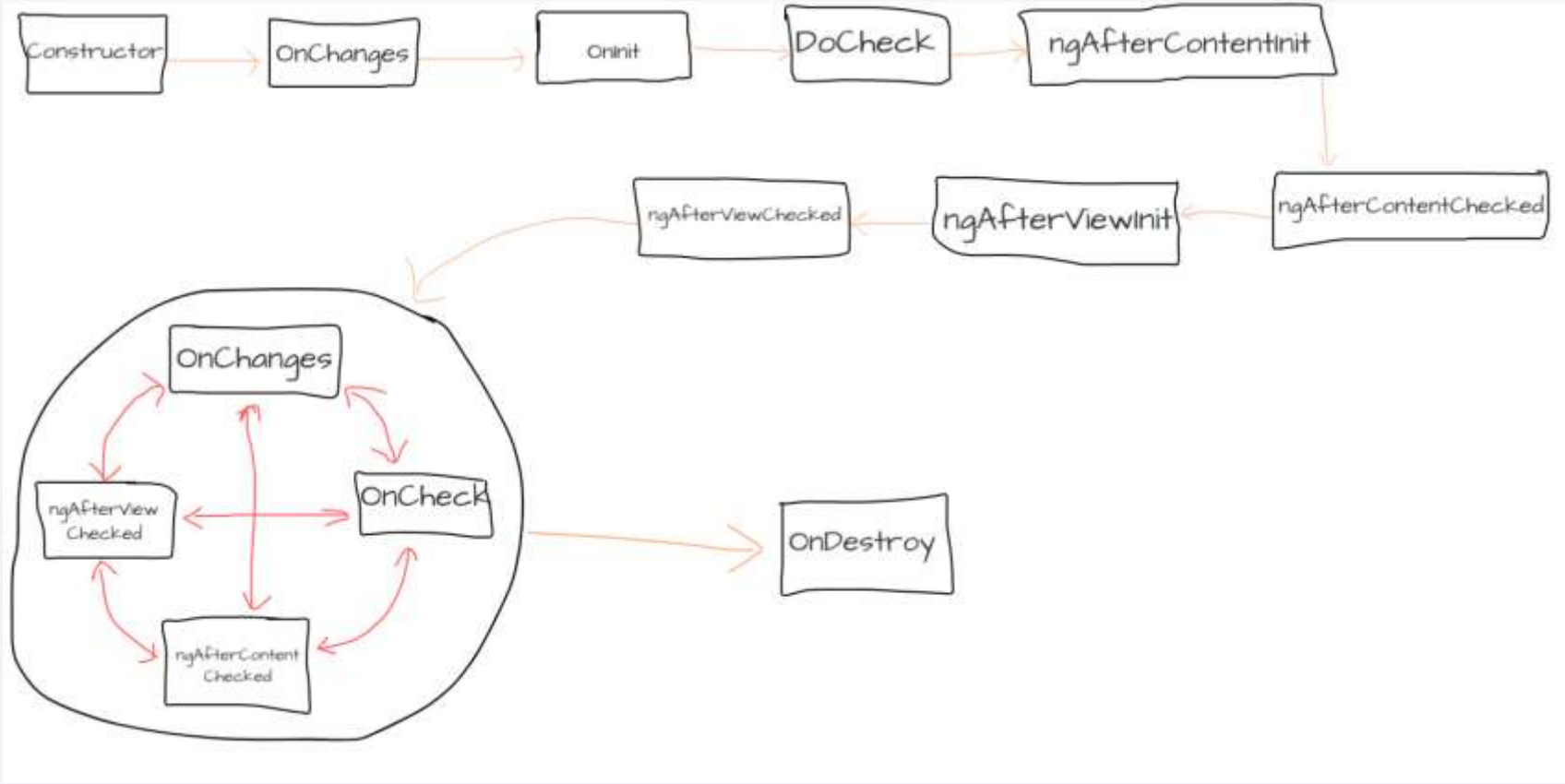


Lifecycle hooks (Components lifecycle)

- A component instance in Angular has a lifecycle that represents a number of different phases it goes through, that starts when Angular instantiates the component class and ends when the component is destroyed.
- The lifecycle includes the change detection, as Angular checks to see when data-bound properties change, and updates both the view and the component instance as needed.
- Respond to events in the lifecycle of a component or directive by implementing one or more of the lifecycle hook interfaces in the Angular core library.
- https://angular.io/guide/lifecycle-hooks#lifecycle-event-sequence



Lifecycle hooks (Components lifecycle)



Trimble

Data binding

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Data binding

Data binding allows to define communication between a component and the DOM.

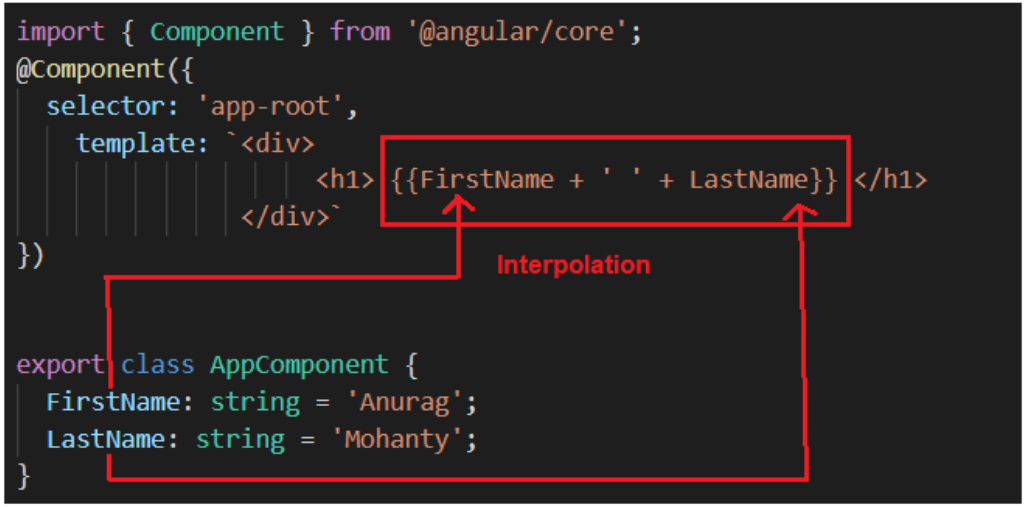
Types:

1. Interpolation: `{{value}}`
2. Property binding: `[property]="value"`
3. Event binding: `(event)="method()"`
4. Two way data binding: `[(ngModel)]="property"`



Interpolation: {{value}}

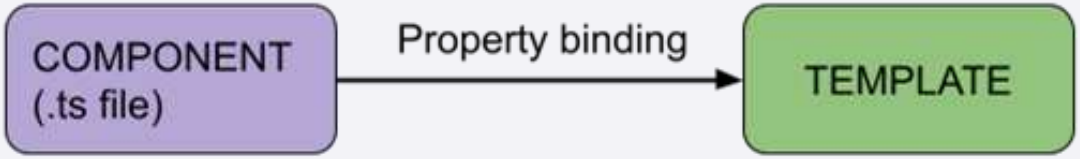
Interpolation refers to embedding expressions into the text between HTML element tags and within attribute assignments. By default, interpolation uses as its delimiter the double curly braces, {{ and }}.



Property binding: [property]=”value”

Property binding flows a value in one direction, from a component's property into a target element property.

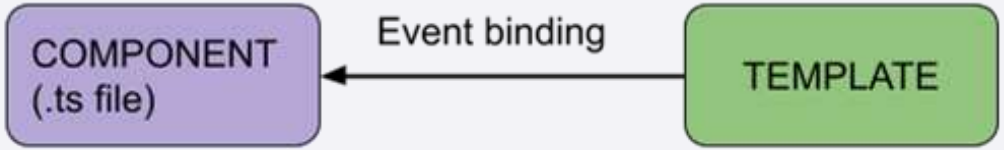
[property]= “value”



Event binding: (event)=”method()”

When an event is triggered, the desired method is called in the component.



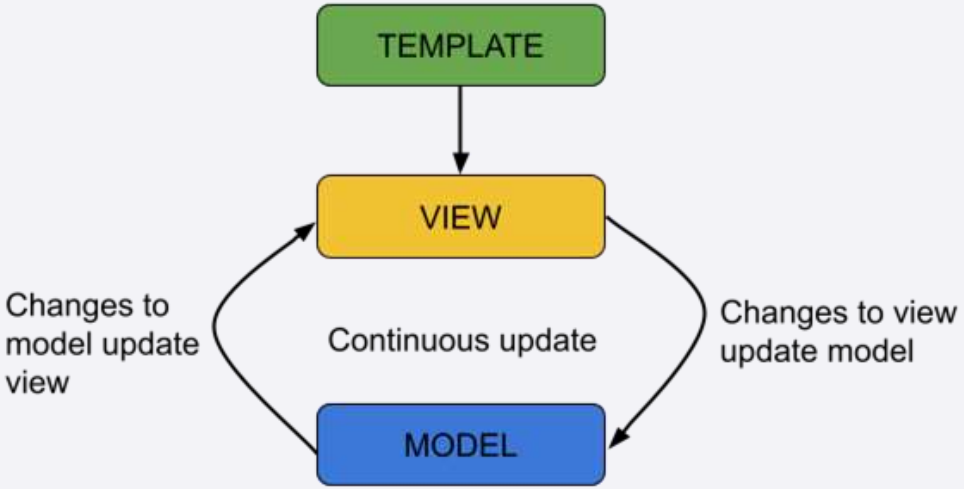


Two way data binding: [(ngModel)]=”property”

Allows bidirectional dataflow.

`[(ngModel)]=`

`“property”`



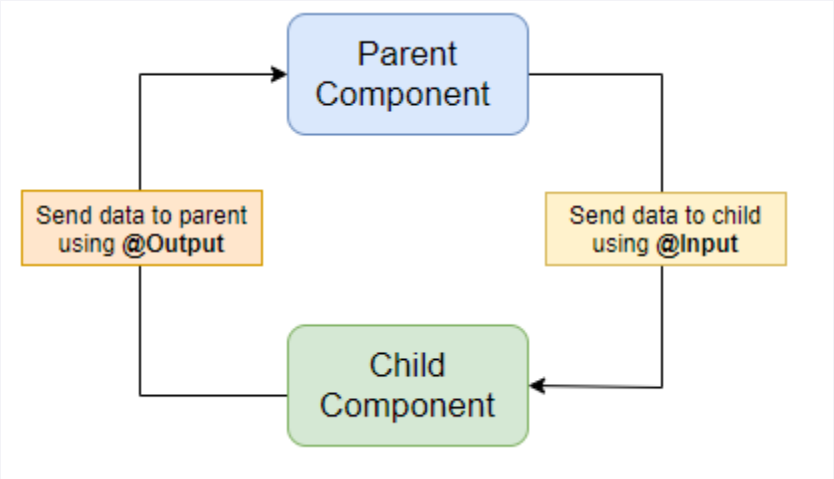
Trimble

@Input() and @Output()

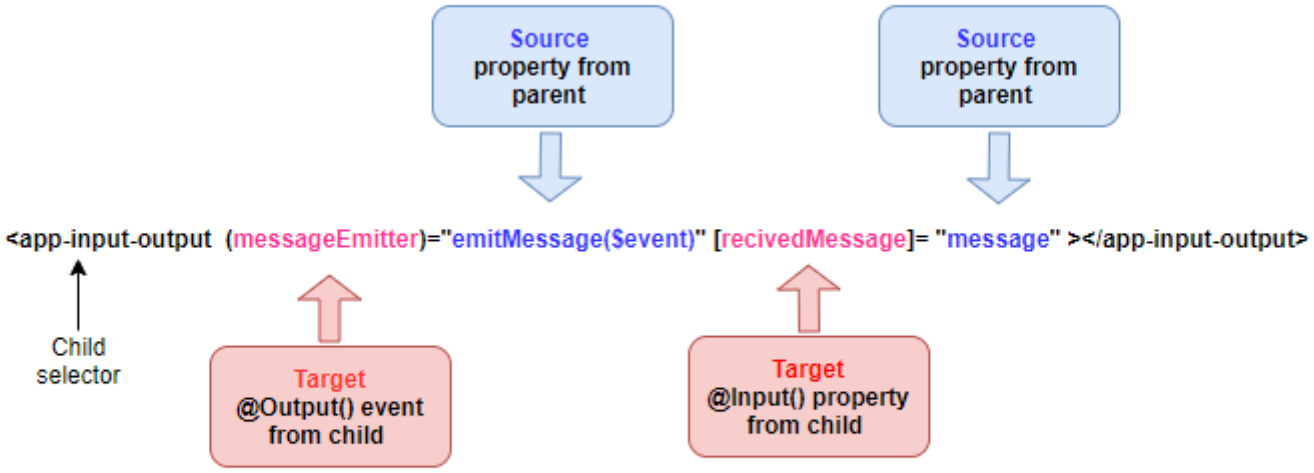
© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

@Input() and @Output()

Represents common component communication scenarios in which two or more components share information.



@Input() and @Output()



@Input

Send a message from parent component to a child component.

- **Child component:** define a simple property that will be the message and add the **@Input** decorator in front of the property

```
@Input() recivedMessage: string;
```

- **Parent component:** declare a property which will contains the message that will be sent in the child component:

```
messageToSent ="Message for ChildComponent";
```

```
<app-child [recivedMessage]="messageToSent"></app-child>
```



@Output

Send a message from a child component to the parent component.

- Child component:
  - Define the message property that will be sent, its type is **EventEmitter**, which means it's an event.
  - Set the type of the data which will be emitted
  - Add the **@Output** decorator in front of the property

```
@Output() messageEventEmitter = new EventEmitter<string>();
```



Pass data from child to parent using @Output

- Parent component:
  - Add a function that will get the message

```
getMessage(message: string) {
  // use message;
}
```

  - Bind the parent's method to the child's event

```
<app-child (messageEventEmitter)="getMessage($event)"></app-child>
```



Pass data from child to parent using @Output

- Create a function that can be called by clicking on a button, which will emit the message to the parent:

```
sendMessageToParent(message: string) {
  this.messageEventEmitter.emit(message)
}
```

```
<button (click)="sendMessageToParent(message)">Send</button>
```



Pipes

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Pipes

Pipes let you declare display-value transformations in your template HTML.



- E.g: `{{interpolated_value | pipe_name}}`
- Angular has defined various pipes: `{{ title | uppercase }}`
- We can use a pipe with parameters: `{{ officialDate | date:"shortTime"}}`
- We can chain pipes: `{{ officialDate | uppercase | date}}`
- https://angular.io/api?type=pipe



Custom Pipes

You can create custom pipes that can manipulate your displayed data in any desired way.

E.g.:

```
import { Pipe, PipeTransform } from '@angular/core';
@Pipe({name: 'addValue'})
export class AddValuePipe implements PipeTransform {
  transform(value: number, addedValue?: number): number {
    return isNaN(addedValue)? value : value + addedValue;
  }
}
```



Directives

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Directives

There are **three kinds of directives** in Angular:

- **Components**—directives with a template.
- **Attribute directives**—change the appearance or behavior of an element, component, or another directive.
- **Structural directives**—change the DOM layout by adding and removing DOM elements.



Attribute Directives

- **Attribute directives**—change the appearance or behavior of an element, component, or another directive.

https://stackblitz.com/angular/ooxrongmbdx



Structural Directives

Structural directives—change the DOM layout by adding and removing DOM elements.

- **ngIf** - makes a DOM element to be added or removed based on a boolean expression

```
<p *ngIf="expression==true">
  Expression is true.
</p>
<p *ngIf="expression==false">
  Expression is false.
</p>
```



Structural Directives

- **ngFor** - makes a DOM element to be added for how many times the boolean expression is true

```
<div *ngFor="let element of array; let i=index; let odd=odd;" >
<p> {{element}} </p>
<p> At index {{i}} </p>
<p> Is index an odd number?: {{odd}} </p>
</div>
```



Commands

- Create a new app: **ng new** *app_name*
- Create a new module: **ng generate** *module_name*
- Create a new component: **ng generate** *component_name*
- Install packages (when needed): **npm install**
- Run the app: **ng serve** (app runs on http://localhost:4200 by default)



Bye

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information



Angular Workshop

Course 4: Angular Routing & Angular Forms

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Trimble

Agenda

- Routing
- Angular Forms



Trimble

Angular Routing

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Angular Routing

In a Single Page Application (SPA), all of your application's functions exist in a single HTML page. As users access your application's features, the browser needs to render only the parts that matter to the user, instead of loading a new page. This pattern can significantly improve your application's user experience.

To define how users navigate through your application, you use routes. Add routes to define how users navigate from one part of your application to another. You can also configure routes to guard against unexpected or unauthorized behavior.



<base href>

- The **<base> tag** specifies the **base URL** and/or target for all **relative URLs** in a document.
- The **router** uses the browser's **history.pushState** for **navigation**. pushState lets you customize in-application URL paths; for example, localhost:4200/first-component.
- You must add a **<base href>** element to the application's index.html for **pushState routing to work** (in the <head> of the index.html).
- The browser uses the <base href> value to prefix relative URLs when referencing CSS files, scripts, and images.

<base href="/">



Angular Routing - module configuration

- AppRoutingModule is the NgModule where routes can be configured.

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';

import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';

...
@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule //module were routes are configured
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

- Create and import the **AppRoutingModule** into **AppModule** and add it to the **imports array**.
- When using Angular CLI, this step can be done automatically using the **--routing flag.**



Angular Routing - routes configuration in 3 steps

Step 1:

- **Import RouterModule and Routes into your routing module (called AppRoutingModule).** When using Angular CLI, this step is performed automatically. The CLI also sets up a Routes array for your routes and configures the imports and exports arrays for @NgModule().

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';


const routes: Routes = []; //sets up routes constant where routes are defined
...
@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```



Angular Routing - routes configuration in 3 steps

Step 2:

- **Define your routes in your Routes array**. Each route is a JavaScript object that contains two properties:
  - **path** - defines the URL path for the route.
  - **component** - defines the component Angular should use for the corresponding path.

```
const routes: Routes = [
  { path: 'first-component', component: FirstComponent },
  { path: 'second-component', component: SecondComponent },
];
```



Angular Routing - routes configuration in 3 steps

Step 3:

- **Add your routes to your application:**
  - **routerLink** directive lets you link to specific routes in your app. Set the routerLink attribute for an anchor tag that defines you route;
  - **routerLinkActive** directive lets you add a CSS class to an element when the link's route becomes active.
- Next, update your component template to include **<router-outlet>**.
- The **Router-outlet** is a **directive** from the router library that is used like a component. Router-outlet acts as a **placeholder** that Angular dynamically fills based on the current router state. So this element informs Angular to update the application view with the component for the selected route.

Code example: https://github.com/IoanaChiriacescu/test-routing



Route order

- **The order of routes is important** because the Router uses a *first-match wins* strategy when matching routes, **so more specific routes should be placed above less specific routes**.
- List routes with a static path first, followed by an empty path route, which matches the default route.
- The wildcard route comes last because it matches every URL and the Router selects it only if no other routes match first.



Route wildcard

- The Angular router selects the **wildcard route** (if one is set up) any time the requested URL doesn't match any router paths.
- A wildcard route is the last route because it matches any URL.

{ path: '', component: <component-name> }**

  - ****** (two asterisks) - indicate to Angular that this routes definition is a wildcard route.
  - **component** property - can be any component in your application. Common choices include the application's main component or an application-specific "page not found" component, which you can define to display a 404 page to your users.



Route redirect

- To set up a redirect, configure a route with the path you want to redirect from, the component you want to redirect to, and a pathMatch value that tells the router how to match the URL.

```
const routes: Routes = [
  { path: 'first-component', component: FirstComponent }, // specific route
  { path: 'second-component', component: SecondComponent }, // specific route
  { path: '',   redirectTo: '/first-component', pathMatch: 'full' }, // redirect to `first-component`
  { path: '**', component: PageNotFoundComponent }, // wildcard route
];
```



Nesting routes

- As your application grows more complex, you might want to create routes that are relative to a component other than your root component. These types of nested routes are called child routes. This means you're adding a second <router-outlet> to your app, because it is in addition to the <router-outlet> in AppComponent.
- A child route is like any other route, in that it needs both a path and a component. The one difference is that you place child routes in a children array within the parent route

Code example: https://github.com/IoanaChiriacescu/test-routing



Route parameters

- **A route with parameter** provides the ability to send additional information to the route.
- The colon **:** character in the path indicates that :id is a placeholder for a specific id.

```
const routes: Routes = [
  { path: 'first-component', component: FirstComponent }, // specific route
  { path: 'second-component/:id', component: SecondComponent }, // route with parameter
];
```

```
<p>
  <a [routerLink]="'/first-component'">Route to the first-component</a>
</p>
<p>
  <a [routerLink]="['/second-component', idParameter]">Route to the second-component with the id parameter</a>
</p>
```



Accessing query parameters and fragments from a route

- **ActivatedRoute** provides access to information about a route associated with a component that is loaded in an outlet.

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute, ParamMap } from '@angular/router';

@Component({
  selector: 'app-second',
  templateUrl: './second.component.html',
  styleUrls: ['./second.component.scss'],
})
export class SecondComponent implements OnInit {
  id: string;
  constructor(private route: ActivatedRoute) {}

  ngOnInit(): void {
    this.route.paramMap.subscribe((params: ParamMap) => {
      this.id = params.get('id'); //id parameter in route
      console.log(this.id);
    });
  }
}
```



Route guards

- To control access to different parts of the application, route guards are used. You may want to control access because:
  - the user is not authorized to navigate to the target component
  - the user must login first
  - you should fetch some data before you display the target component
  - you might want to save pending changes before leaving a component
  - you might ask the user if it's okay to discard pending changes rather than save them



Route guards

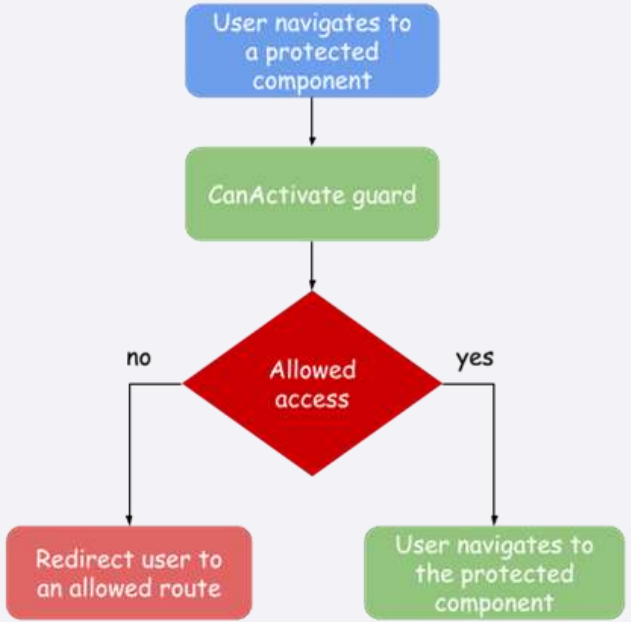
- The following route guards are available in Angular:
  - CanActivate
  - CanActivateChild
  - CanDeactivate
  - Resolve
  - CanLoad

Examples: https://angular.io/guide/router-tutorial-toh#milestone-5-route-guards



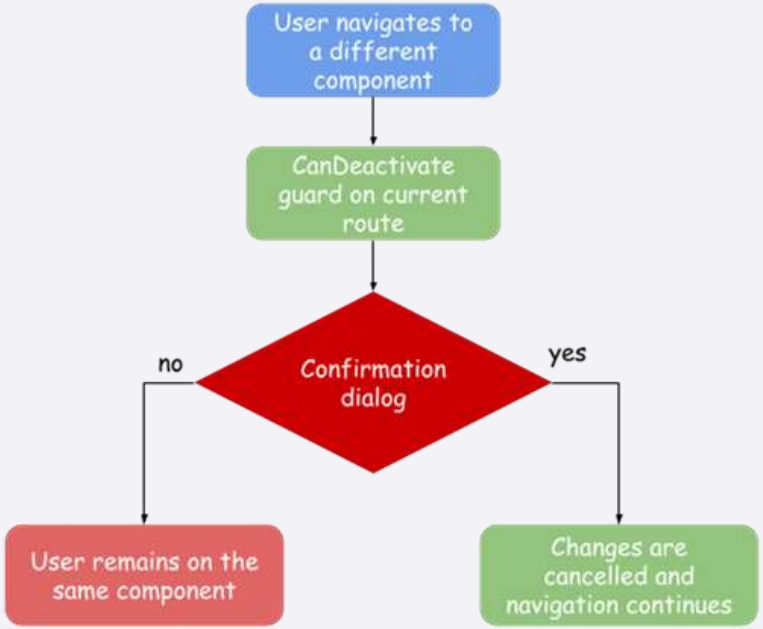
Route guards - CanActivate

- The **CanActivate** guard is the tool to manage navigation rules like:
  - the user can access a feature based on his role;
  - the user access is permitted only for authenticated;
  - block or limit access until the user's account is activated.



Route guards - CanDeactivate

- The **CanDeactivate** guard is the tool to manage what to do with unsaved changes and how to proceed. For example:
  - when the user navigates to a different component, you can let the user decide what to do with unsaved changes;
  - when a server call is done, you need to stop the navigation while you wait, asynchronously, for the server response.



Route guards - Resolve

- The **Resolve** guard (**resolver**) is the tool to delay rendering the routed component until all necessary data has been fetched. For example:
  - when using a real API, you might want to not render the page until all data has been fetched so you don’t display a blank component while waiting for the data;
  - you can pre-fetch data from the server using a resolver so it's ready the moment the route is activated.



Lazy loading modules

- By default, modules are eagerly loaded, which means that all the modules are loaded as soon as the application loads, whether or not they are immediately necessary.
- **Lazy loading** modules means that Angular only loads modules as needed, rather than loading all modules when the application launches.



Trimble

Angular Forms

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Angular Forms

- Applications use forms to enable users to log in, to update a profile, to enter sensitive information, and to perform many other data-entry tasks.
- Angular provides two different approaches to handling user input through forms:
  - **reactive;**
  - **template-driven**.

Both capture user input events from the view, validate the user input, create a form model and data model to update, and provide a way to track changes.

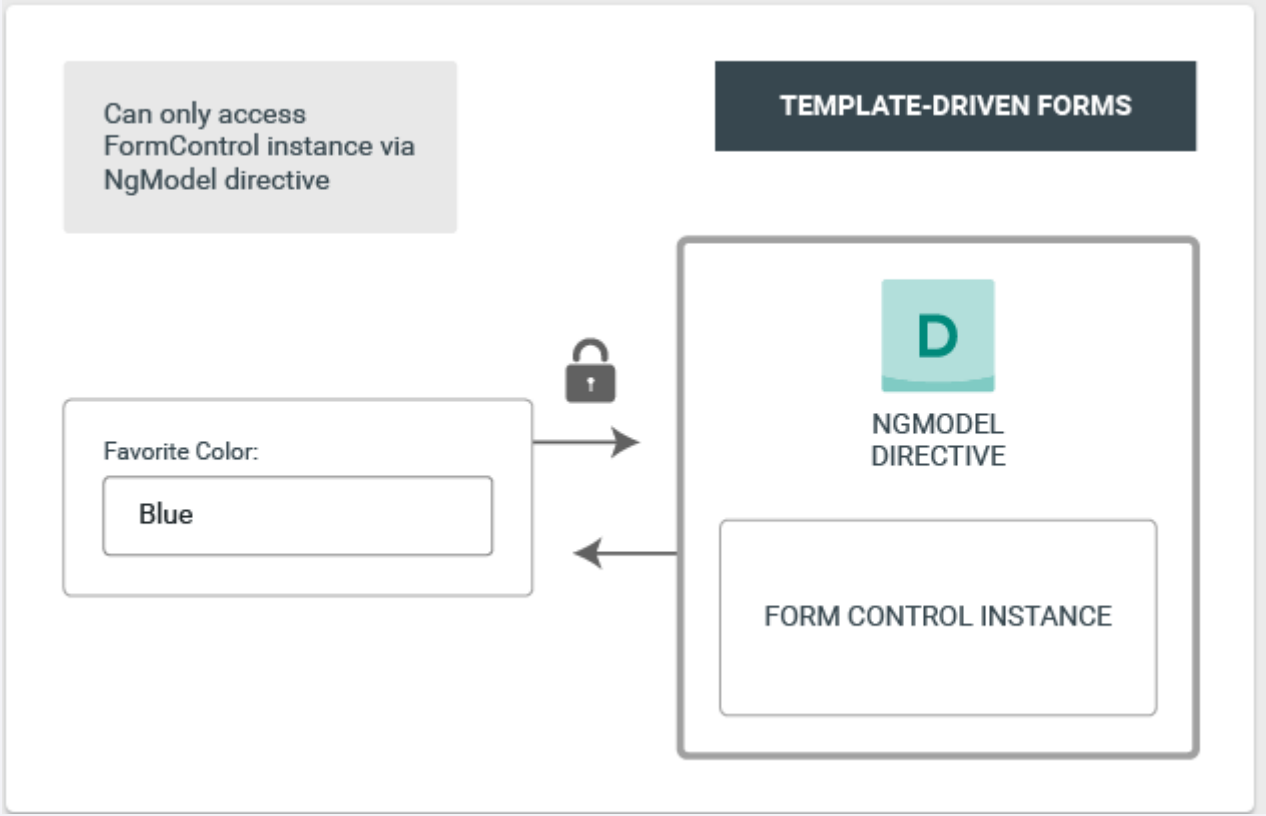


Angular Forms - Template-driven forms

- the directive NgModel creates and manages a FormControl instance for a given form element;
- form element is linked to a directive that manages the model internally;
- the source of truth is the template (means that you do not have direct programmatic access to the FormControl instance);
- the model is always changed to its new value (rely on mutability)
- Step by step example: https://angular.io/guide/forms#building-a-template-driven-form



Angular Forms - Template-driven forms

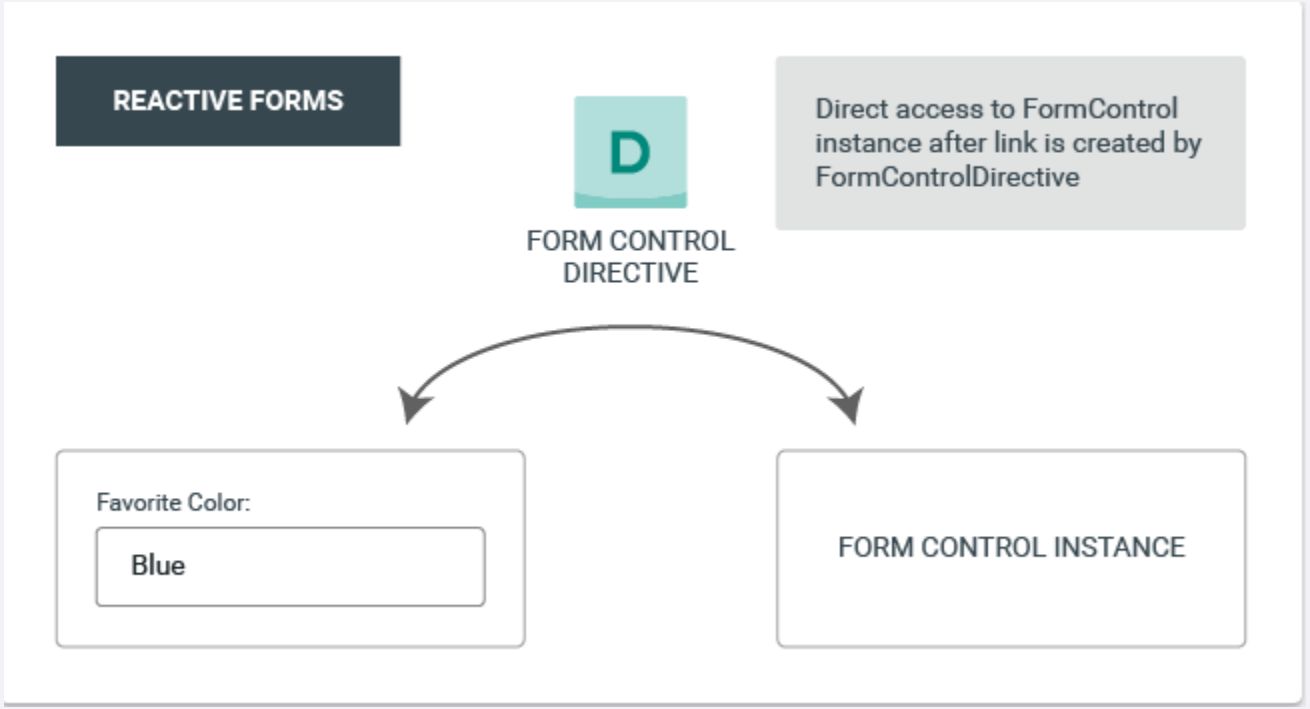


Angular Forms - Reactive forms

- each form element in the view is directly linked to the form model (a FormControl instance)
- the FormControl instance is created explicitly in the component template;
- the form model is the source of truth so you have access to the value and status of the form element at any given point in time
- the FormControl instance always returns a new value when the control's value is updated
- Official documentation https://angular.io/guide/reactive-forms#reactive-forms



Angular Forms - Reactive forms



Angular Forms - Key differences

	Template-driven form	Reactive form
Setup of form model	Implicit, created by directives	Explicit, created in component class
Data model	Unstructured and mutable	Structured and immutable
Data flow	Asynchronous	Synchronous
Form validation	Directives	Functions



Angular Forms - Validation

- Validators are used to manage any set of forms. Angular provides a set of built-in validators as well as the ability to create custom validators.
- To add validation to a **template-driven form**, you add the same validation attributes as you would with native HTML form validation (check validators here). Angular uses directives to match these attributes with validator functions in the framework.
- In a **reactive form**, the source of truth is the component class. Instead of adding validators through attributes in the template, you add validator functions directly to the form control model in the component class. Angular then calls these functions whenever the value of the control changes (check validators here).



Bye

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information



Angular Workshop

Course 5: Services. Dependency injection. Observables

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Trimble

Agenda

- Services
- Dependency injection
- Observables



Services

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

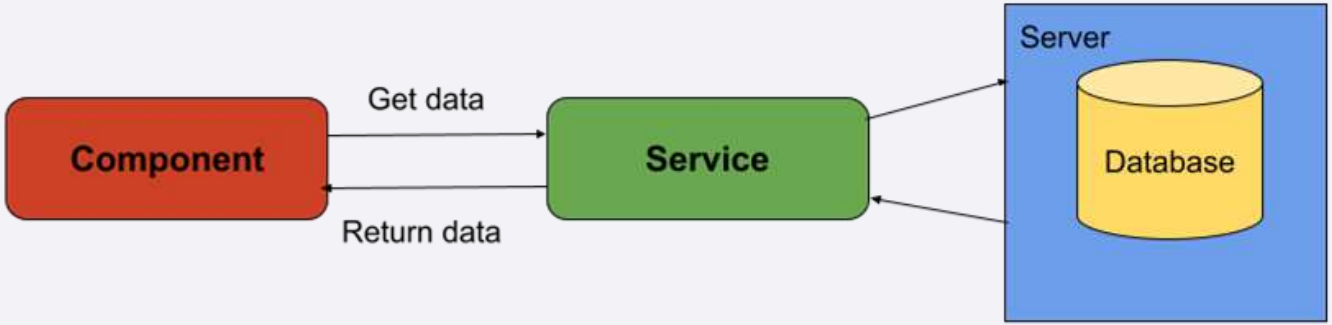
Services

- A **service**'s job is to provide data to a component or to handle data from a component.
- A service is a class that should do something specific and do it well.
- Components vs services:
  - **Components** have view-related functionalities, while **services** have data processing functionalities that can be reused.



Services

- A component can delegate certain tasks to services, such as fetching data from the server or validating user input.
- By defining such processing tasks in an injectable service class, you make those tasks available to any component.



Create a service

- Command: **ng generate service <name>**
- To define a class as a service in Angular, use the **@Injectable()** decorator to provide the metadata that allows Angular to inject it into a component as a dependency.

TODO live code example



Trimble

Dependency injection

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Dependency injection

- Dependency Injection (DI) is a design pattern and mechanism for creating and delivering some parts of an application to other parts of an application that require them.
- Dependency injection is a fundamental concept in Angular.
- DI is a part of the Angular framework which allows classes, such as Components, Directives, Pipes, and Injectables, to configure the dependencies they need.
- In Angular, dependencies are typically services, but they also can be values, such as strings or functions.



Dependency injection

- The **@Injectable()** decorator defines a class as a service in Angular and allows Angular to inject it into a component as a dependency.
- The **injector** is the main mechanism. Angular creates an application-wide injector during the bootstrap process. You don't have to create injectors. An injector creates dependencies and maintains a container of dependency instances that it reuses.
- A **provider** is an object that tells an injector how to obtain or create a dependency.



Dependency injection

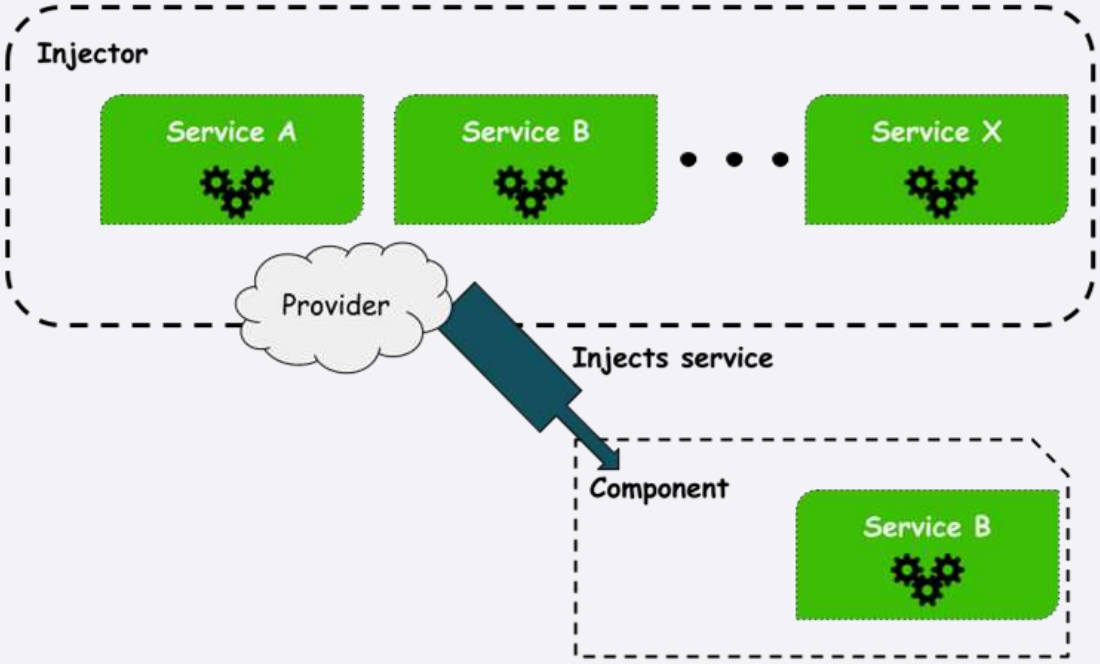
- When Angular creates a new instance of a component class, it determines which **services** or other **dependencies** that component needs by looking at the **constructor** parameter types.

```
constructor(private service: CustomService) { }
```



Dependency injection

- When Angular discovers that a **component depends on a service**, it first checks if the injector has any existing instances of that service.
- If a **requested service** instance **doesn't yet exist**, the injector **makes one** using the registered provider, and adds it to the injector before returning the service.



Dependency injection

- When **all requested services have been resolved** and returned, Angular can **call the component's constructor** with those services as arguments.



Providing services

Three types of providing services:

- At application(root) level
- At module level
- At component level



Providing dependency

- Providing services at application level:

  When you provide the service at the root level, Angular creates a single, shared instance of that service and injects it into any class that asks for it.

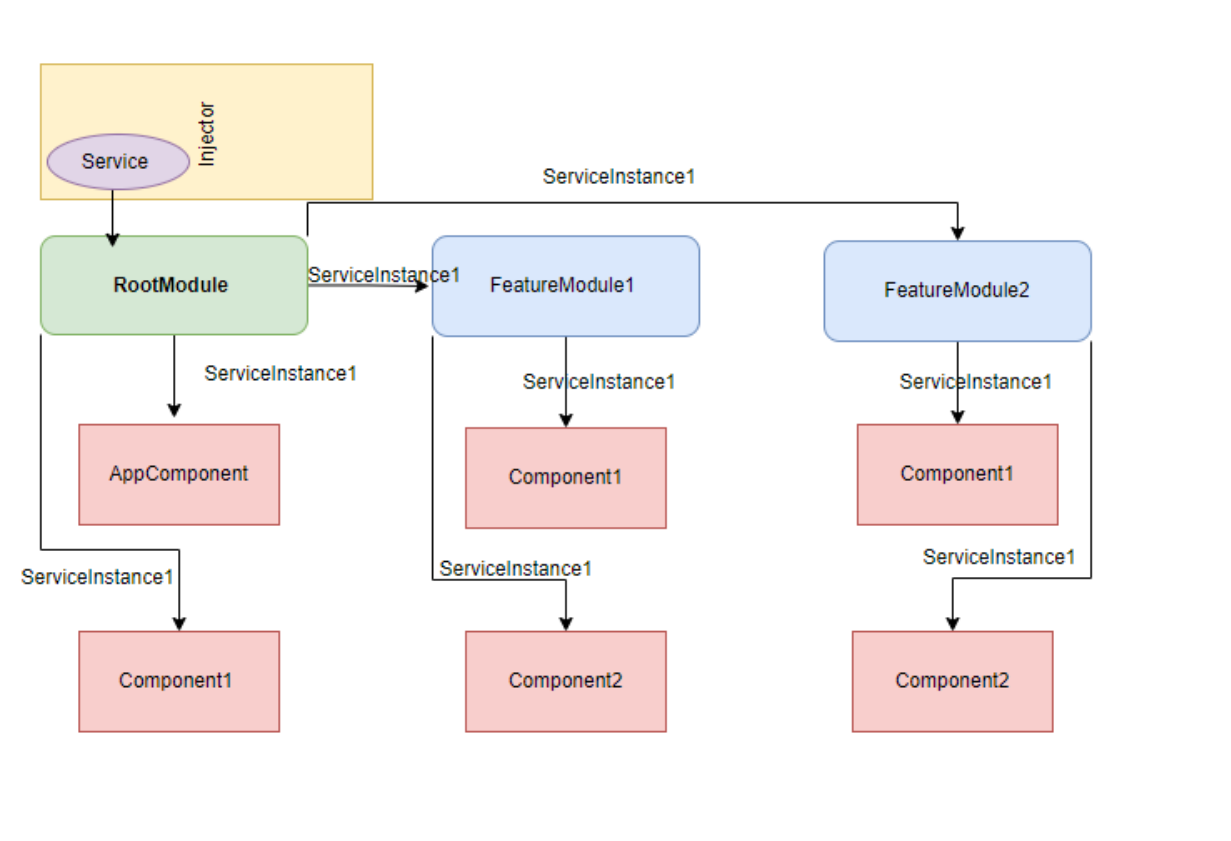
```
@Injectable({

 providedIn: 'root',

})
```



Providing dependency



Providing services

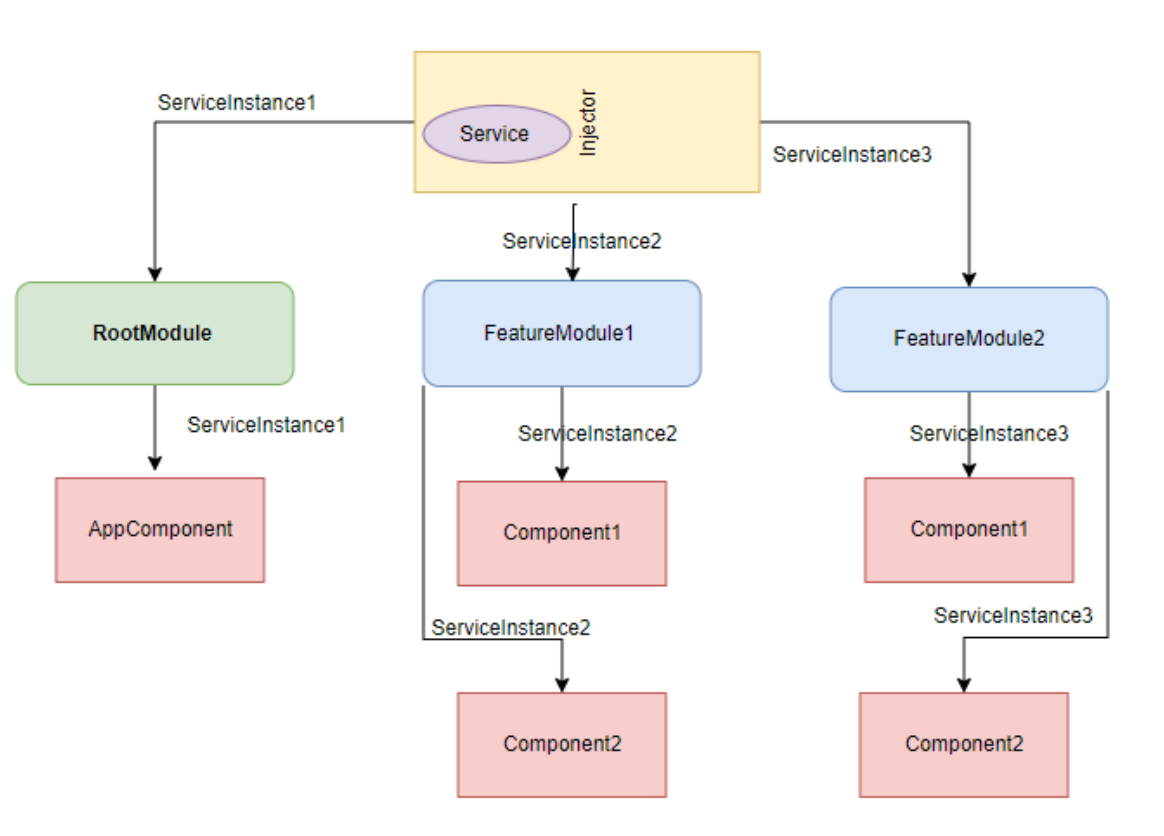
- Providing services at module level:

  When you register a provider with a specific NgModule, the same instance of a service is available to all components in that NgModule.

```
@NgModule({
  providers: [
  CustomService
 ],
 ...
})
```



Providing services



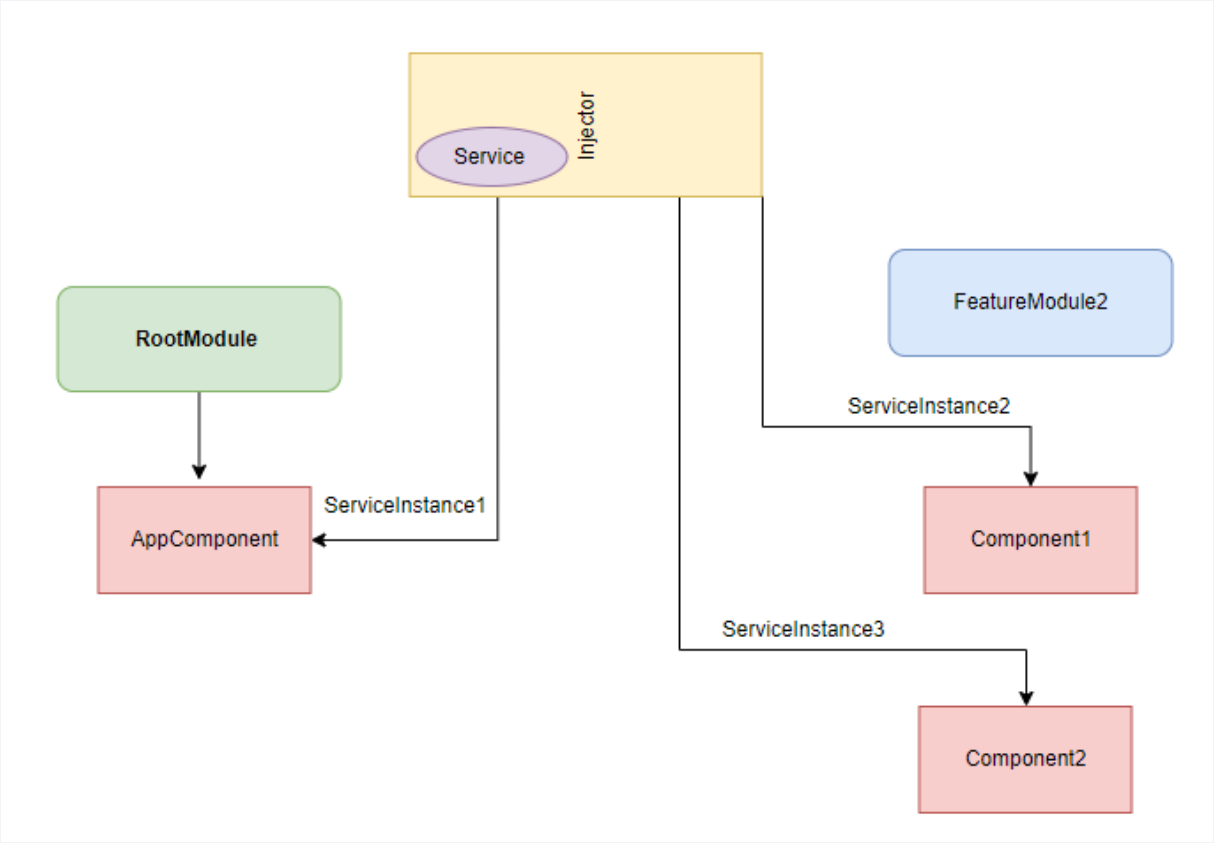
Providing services

- Providing services at component level:
  - When you register a provider at the component level, you get a new instance of the service with each new instance of that component.

```
@Component({
  selector:    'component-selector',
  templateUrl: './component.html',
  providers:   [ CustomService ]
})
```



Providing dependency



Observables

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Observables

- **Observables** are used frequently in Angular and are a technique for event handling, asynchronous programming, and handling multiple values.
- The **Observable** pattern is a design pattern in which an object, called the subject, keeps track of other objects, called **observers**, and notifies them about state changes.

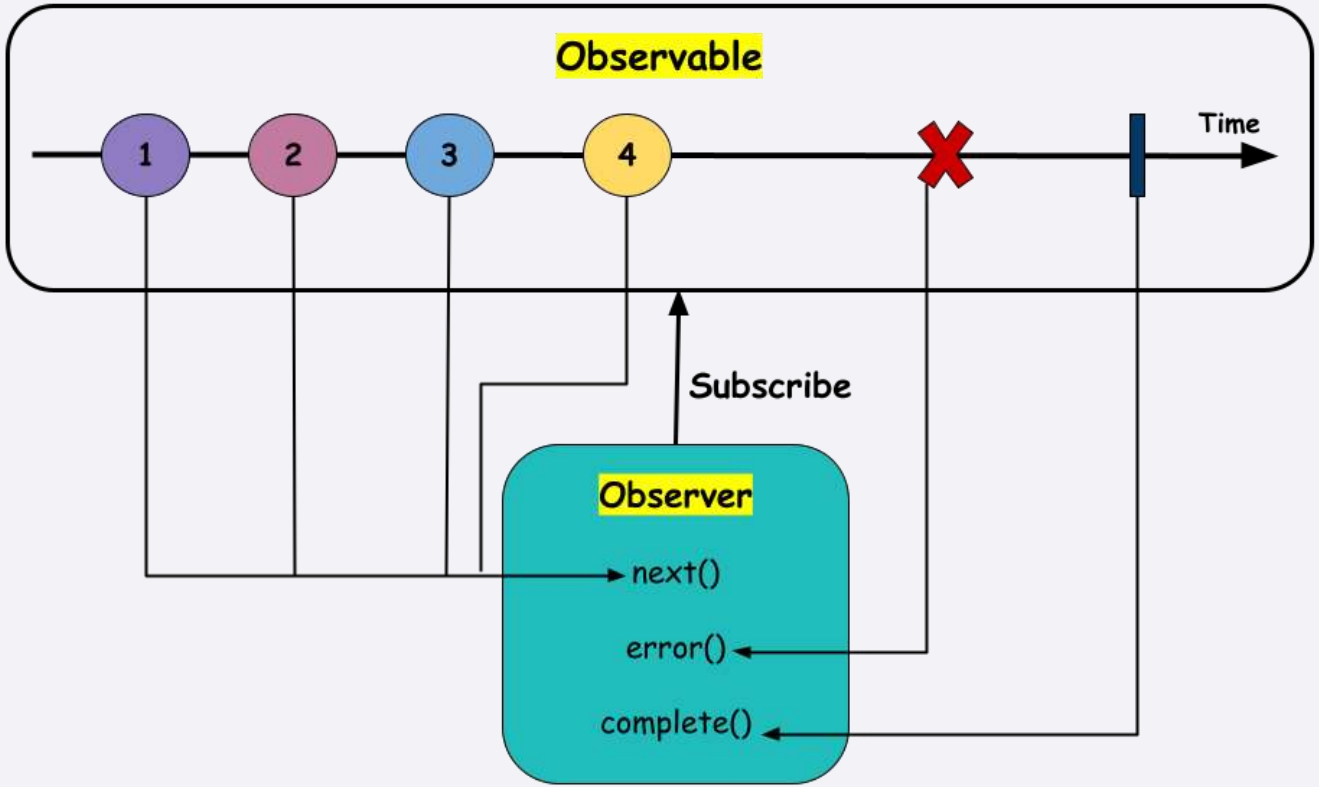


Observables

- **Observable** emits the value from the stream **synchronously or asynchronously**. Also, it emits the complete signals when the stream completes or an error signal if the stream throws an error.
- **Observables** are **declarative**, so a defined function for publishing values it is not executed until a consumer **subscribes** to it. The subscribed consumer then receives notifications until the function completes, or until they **unsubscribe**.



Observables



Observables

- Subscribing to an **Observable** is analogous to calling a Function.
- **Observables** are able to deliver values either synchronously or asynchronously
- **Observables** can "return" multiple values over time, something which functions cannot.
- **Observers** are just objects with three callbacks, one for each type of notification that an Observable may deliver.



RxJS library

- **RxJS** is a library for composing asynchronous and event-based programs by using observable sequences. It provides one core type, the Observable, satellite types (Observer, Schedulers, Subjects) and operators inspired by Array methods (map, filter, reduce, every, etc) to allow handling asynchronous events as collections.

Check RxJS documentation here



RxJS library

- RxJS concepts:
  - **Observable** represents the idea of an invokable collection of future values or events.
  - **Observer** is a collection of callbacks that knows how to listen to values delivered by the Observable.
  - **Subscription** represents the execution of an Observable.
  - **Operators** are pure functions that enable a functional programming style of dealing with collections with operations like map, filter, concat, reduce, etc.



RxJS - Operators

- Operators are functions. There are two kinds of operators: pipeable and creation.
- A Pipeable Operator is a function that takes an Observable as its input and returns another Observable. It is a pure operation: the previous Observable stays unmodified.
- Creation operators are functions that can be used to create an Observable with some common predefined behavior or by joining other Observables.

Check operators here



Bye

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information



Angular Workshop

Course 6: Communication with server. HTTP

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Trimble

Agenda

- Communication with server
- HTTP
- Types of HTTP requests
- HTTP Status Codes
- HTTPClient



Communication with server

What do we need in order to achieve that?



Communication with server

There are several ways to create a communication between client and server. They all consist in creating a transfer protocol and then, all the communications will follow the protocol rules.









HTTP

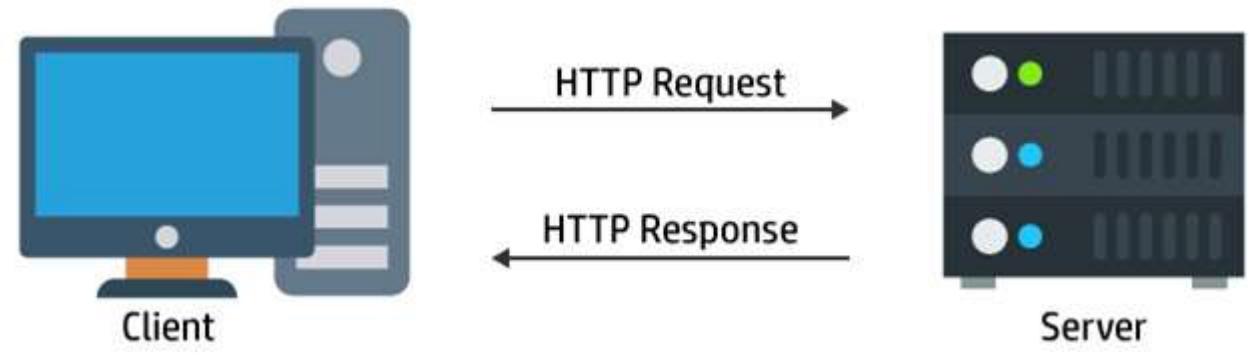
The **Hypertext Transfer Protocol (HTTP)** is an application protocol for distributed, collaborative, hypermedia information systems.

HTTP is the foundation of data communication for the World Wide Web, where hypertext documents include hyperlinks to other resources that the user can easily access



How HTTP works

The client sends **requests** to the server and the server sends **responses**.



HTTP Requests and HTTP Responses

HTTP request is a message sent by the client to initiate an action on the server.

HTTP response is a response from a server to a client. The aim of the response is to provide the client with the resource it requested



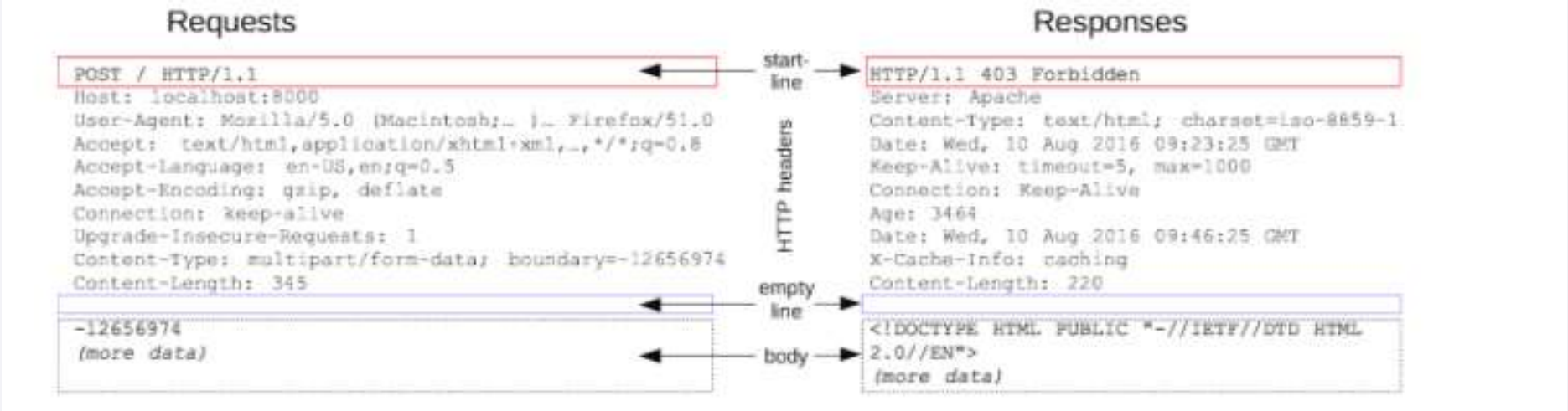
HTTP message

The request message consists of the following:

- a request/response line (e.g. GET /images/logo.png, HTTP/1.1 200 OK)
- request/ response header fields (e.g., Accept-Language: en, Content-Type: text/html).
- an empty line
- an optional message body



HTTP message





HTTP types of requests

- **GET**

The GET method requests a representation of the specified resource. Requests using GET should only retrieve data and should have no other effect.

- **HEAD**

The HEAD method asks for a response identical to that of a GET request, but without the response body. This is useful for retrieving meta-information written in response headers, without having to transport the entire content.



HTTP types of requests

- **POST**

The POST method requests that the server accept the entity enclosed in the request as a new subordinate of the web resource identified by the URI.

- **PUT**

The PUT method requests that the enclosed entity be stored under the supplied URI. If the URI refers to an already existing resource, it is modified; if the URI does not point to an existing resource, then the server can create the resource with that URI.



HTTP types of requests

- **DELETE**

  The DELETE method deletes the specified resource.

- **TRACE**

The TRACE method echoes the received request so that a client can see what (if any) changes or additions have been made by intermediate servers.



HTTP types of requests

- **OPTIONS**

The OPTIONS method returns the HTTP methods that the server supports for the specified URL. This can be used to check the functionality of a web server by requesting '*' instead of a specific resource.

- **PATCH**

The PATCH method applies partial modifications to a resource.



HTTP status codes

As we saw earlier, every http response has a **status code**. It is a 3 digit number.

Status codes are grouped by the type of response as it follows:

1. Informational 1XX
2. Successful 2XX
3. Redirection 3XX
4. Client Error 4XX
5. Server Error 5XX



HTTP status codes

A full list of status codes can be found here:

https://en.wikipedia.org/wiki/List_of_HTTP_status_codes

https://developer.mozilla.org/en-US/docs/Web/HTTP/Status



HTTPClient

HTTPClientModule is an Angular module that contains **HTTPClient** service. This service provides methods of interacting with an API that uses HTTP.

The **HTTPClient** service can be injected as a dependency of an application class.

The multiple versions of the Angular HTTP module all have an **RxJS Observable-based API**. This means that the multiple calls to the HTTP module will all return an observable, that we need to subscribe to one way or the other.



Requesting data from a server

HttpClient.get() is a asynchronous method that send an HTTP request to the server and returns an Observable that emits the requested data when the response is received.

This method takes two parameters:

- the server endpoint URL
- options object which is used to configure the request :
  - *observe* option specifies how much of the response to return
  - *responseType* specifies the format in which to return data
  - *headers* allows to set values for request headers (e.g: set the authentication token)



Requesting data from a server

The get method and all http methods from HttpClient don't begin the request until the `subscribe()` is called on the observable that method return.

Each `subscribe()` triggers a separate execution of the observable. So if we subscribe two times we will made two requests.

We can specify the response object type for get method.

```
this.http.get<Config>(this.configUrl);
```



Handling request errors

When the request fails on the server, HttpClient returns an error object that can be handle.

There are two type of errors which can occur:

- from the server backend, which return a HTTP response with a status code (400, 404, 500)
- from the client side, which return 0 status code

RxJS has support for retrying a failed request (retry() operator)



Sending data to a server

HttpClient.post() is used for sending data to server. This method has like parameters :

- server endpoint URL
- body : data which is sent to the server
- options

Post method have a type parameter which you can use to specify that you expect to server to return.

Other methods from HttpClient: **delete(), put().**



Bye

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information



.NET Workshop

Course 7: Introduction. Routing. Formatters and Content Negotiation

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Trimble

Prerequisites

See .NET Core Workshop prerequisites with download links here

- Visual Studio Community 2022(Version >= 17.0)
- .NET 6
- Postman (optional)
- Mongo DB and NoSqlBooster (optional)

Why .NET?

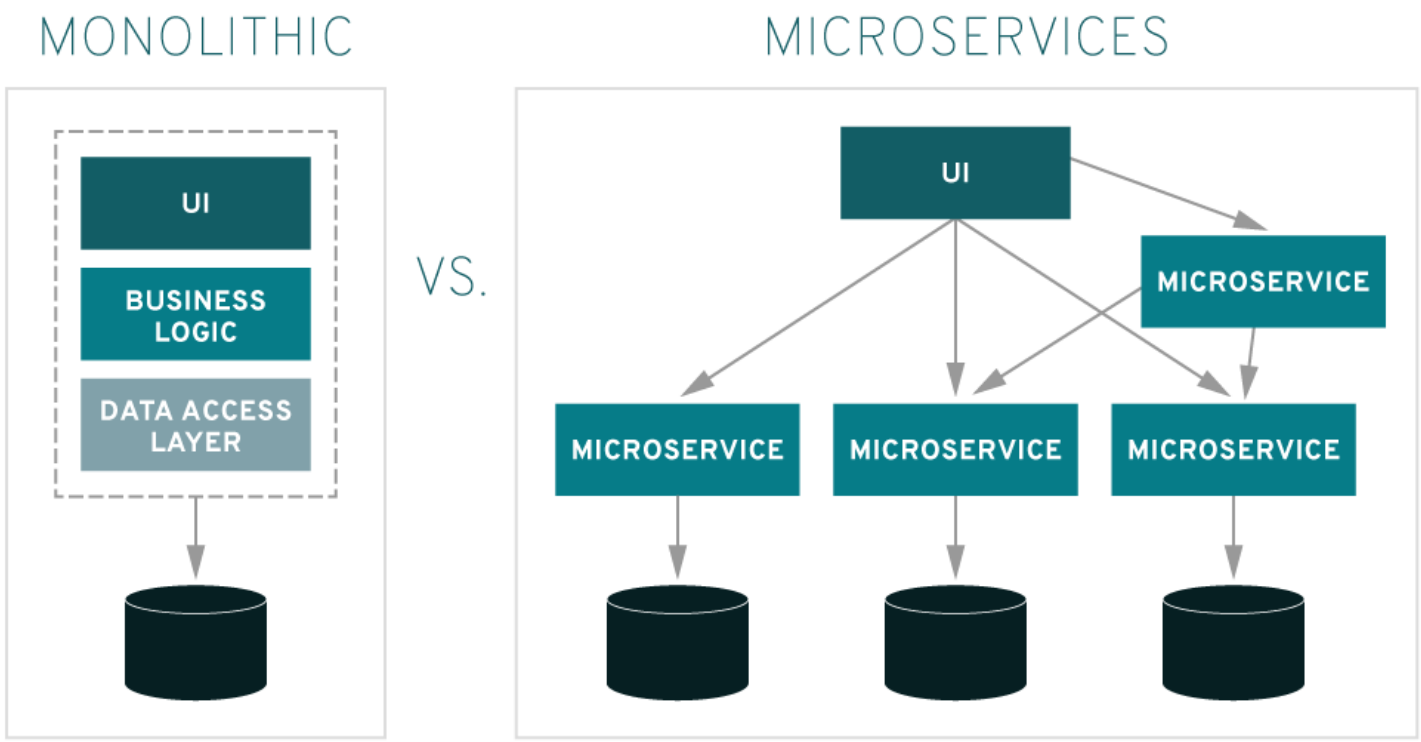

Complete Web Server Independence
Cross-Platform Development
Built-In Dependency Injection
Open Source and Community Focused
Side-by-Side App Versioning
Unified Stack for MVC and Web API
Rapid Development
Better Portability
Rich Controls
Better Security
More Scalable
More Maintainable

Why Microservices?

Also known as microservice architecture provides the ability to structure an application as a set of independent services that are:

- Highly maintainable and testable
- Loosely coupled
- Independently deployable
- Owned by a small team

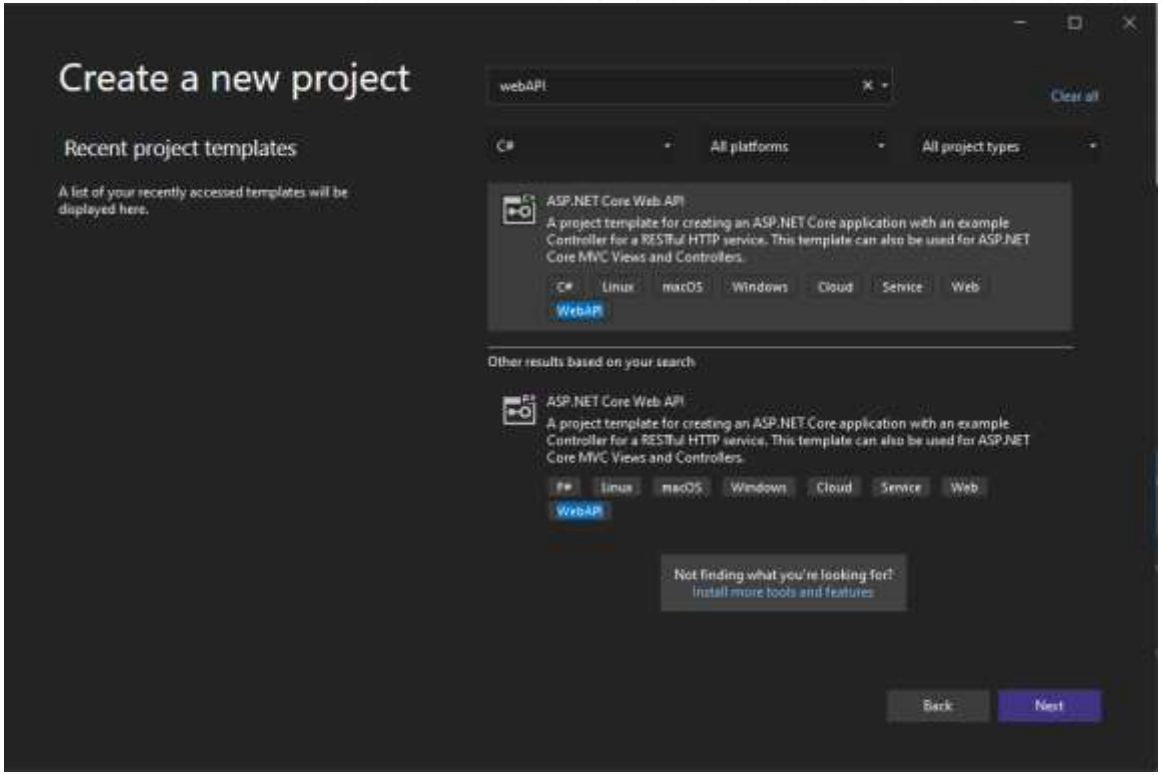
Why Microservices?



How to create a new .NET Core project?

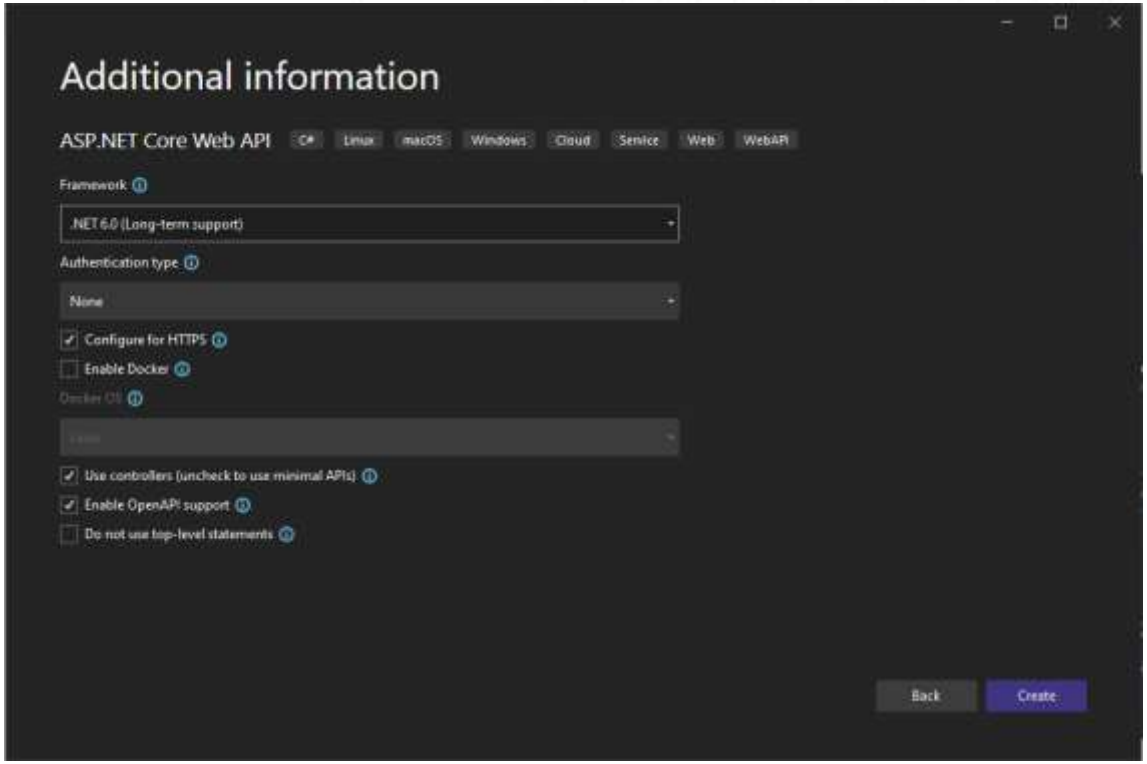
In VS go to File tab and choose New->Project.

Select the ASP.NET Core Web API



How to create a new .NET Core project?

Choose .NET 6.0



Trimble.

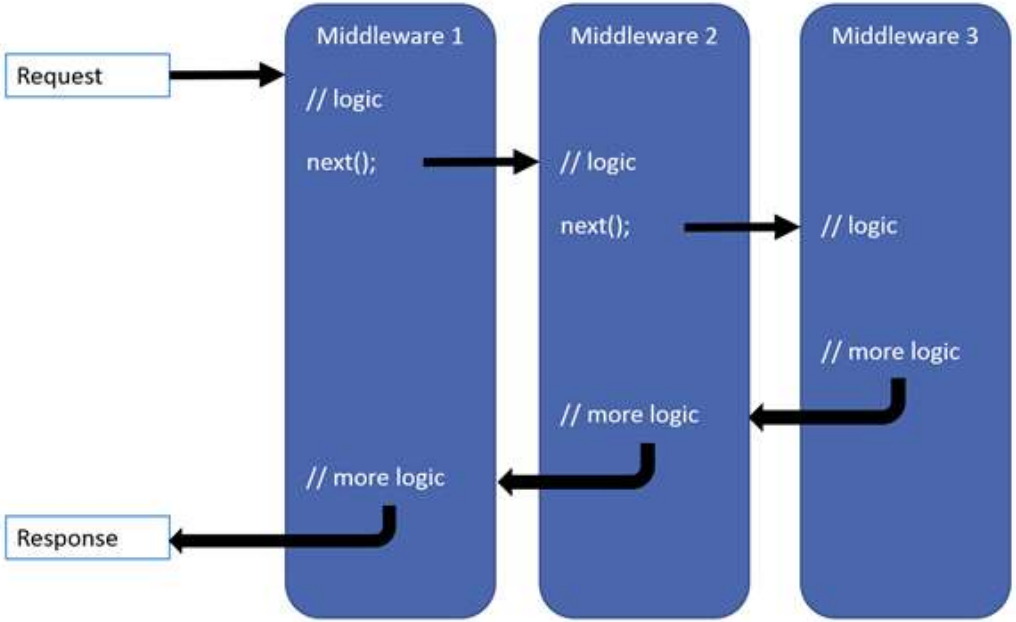
.NET Core Web App Project Structure

Program class

- **Program** class is the entry point of the application.
- Is similar to a console application.
- Runs the web application based on a web host configured by the user.
- Configure the services container used for Dependency Injection (all the services which are used in the application).
- Configure the HTTP request pipeline (how the application responds to individual requests using middleware components).

.NET Core Web App Project Structure

Request pipeline diagram



.NET Core Web App Project Structure

Configuration providers - appsettings.json

- ASP.NET Core provides a configuration that gets settings as name-value pairs from a set of configuration providers. There are built-in configuration providers for a variety of sources: .json files, .xml files, environment variables or custom configuration providers.
- For example, a configuration can come from **appsettings.json** and **environment variables**. When a value is requested, the framework looks first in the appsettings.json file. If the value is found there but also in an environment variable, the value from the environment variable would take precedence.

.NET Core Web App Project Structure

Configuration providers - Environment variable

- Environment variable indicates the runtime environment in which an application is currently running and the specific behaviour.
- ASP.NET Core uses an environment variable called ASPNETCORE_ENVIRONMENT to indicate the runtime environment like Development, Staging or Production.
- Environment properties can be set for a project in the Properties->Debug tab and it will be stored in launchSettings.json file which will be used on local machine.

HTTP Context

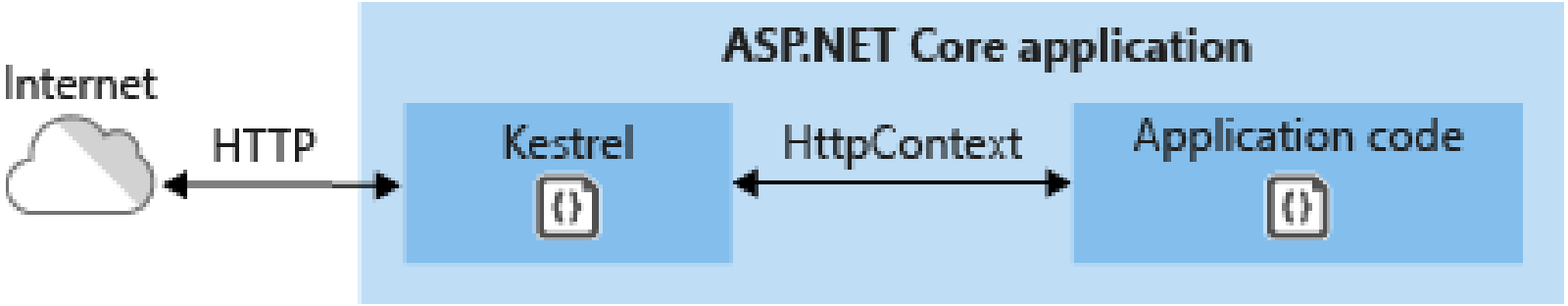
- An ASP.NET Core app runs with an in-process HTTP server implementation. The server implementation listens for HTTP requests and encapsulates all the HTTP-specific information about a single HTTP request into an HttpContext.
- When an HTTP request arrives at the server, the server processes the request and builds an HttpContext object. This object represents the request which your application code can use to create the response.
- https://learn.microsoft.com/en-us/aspnet/core/fundamentals/http-context?view=aspnetcore-7.0

Kestrel

- Kestrel web server is a cross-platform open-source web server for ASP.NET Core.
- Kestrel is the web server that's included and enabled by default in ASP.NET Core project templates.

Kestrel

- The Kestrel Web Server is used to directly processes the incoming HTTP request from the client. In the case of the Kestrel web server, the process name that is used to host and run the ASP.NET Core application is the project name.

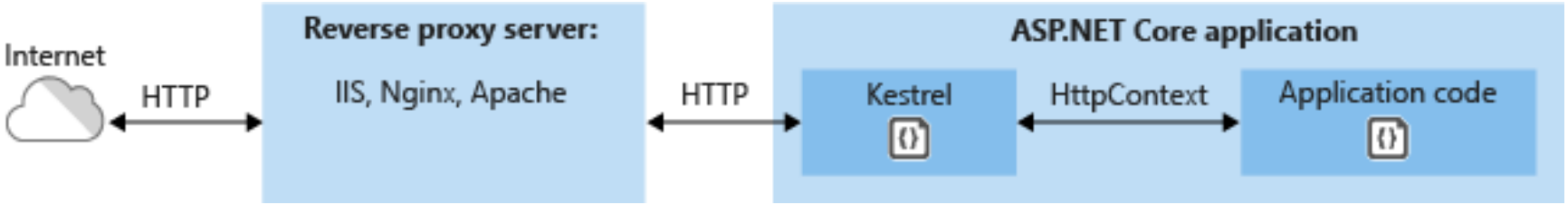


Kestrel with a reverse proxy

- Kestrel can be used by itself or with a reverse proxy server. A reverse proxy server receives HTTP requests from the network and forwards them to Kestrel. Examples of a reverse proxy server include:
  - Internet Information Services (IIS)
  - Nginx
  - Apache
  - YARP: Yet Another Reverse Proxy

Kestrel with a reverse proxy

- A reverse proxy:
  - Can limit the exposed public surface area of the apps that it hosts.
  - Provide an additional layer of configuration and defense.
  - Might integrate better with existing infrastructure.
  - Simplify load balancing and secure communication (HTTPS) configuration.



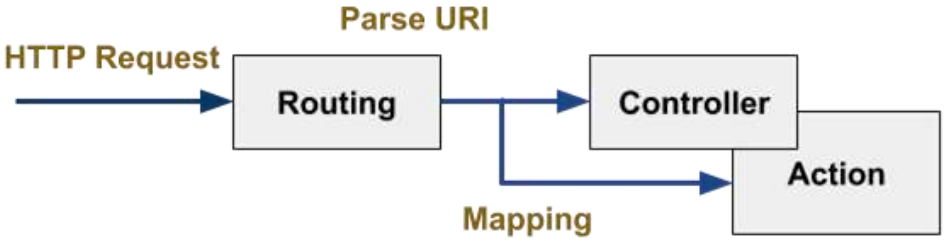
Trimble

Routing
Formatters and Content Negotiation

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Routing

- Controller uses the Routing Middleware to match the URI of an incoming request to a given method in the controller.
- Once a HTTP request is send, the framework parses the URI and map it to the corresponding controller and method of the controller.
- Two types: convention-based routing & attribute-based routing.



Routing

Routing uses a pair of middleware registered by **UseRouting** and **UseEndpoints** in Program class:

- **UseRouting** adds route matching to the middleware pipeline. This middleware looks at the set of endpoints defined in the app, and selects the best match based on the request.
- **UseEndpoints** adds endpoint execution to the middleware pipeline and it runs the delegate associated with the selected endpoint.

Convention-based routing

- Conventional routing is usually used with controllers and views and not for API's.
- It is called conventional routing because it establishes a convention for URL paths based on route template.
- Example:

```
app.UseEndpoints(endpoints =>
  {
     endpoints.MapControllerRoute(
        name: "default",
        pattern: "{controller=Home}/{action=Index}/{id?}");
  });
```

Attribute-based routing

- Is used in API's projects in order to map the application functionalities to a set of resources.
- This type of routing is configured in Startup class by calling **MapControllers()** inside **UseEndpoints** middleware.
- Each **action** in controller class is mapped to a resource using a set of attributes containing the **HTTP verb templates**.

Attribute-based routing

HTTP Method	Attribute	Route template level	Example URI
GET	**HttpGet**	Action	/api/announcements /api/announcements/{id}
POST	**HttpPost**	Action	/api/announcements
PUT	**HttpPut**	Action	/api/announcements/{id}
PATCH	**HttpPatch**	Action	/api/announcements/{id}
DELETE	**HttpDelete**	Action	/api/announcements/{id}
--	**Route**	Controller	Used to add a prefix for all templates of the action level attributes

Route name convention

A resource name in a URI should be represented by a noun which refers to a thing. For example:

- Get request to return all announcements: api/announcements and not api/getannouncements

Binding source attribute for a request

- Binding source attribute defines the location at which an action parameter's value is found. The following binding source attributes exist:
  - [FromBody] - Request body
  - [FromQuery] - Request query string parameter
  - [FromForm] - Form data in the request body
  - [FromHeader] - Request header
  - [FromRoute] - Route data from the current request
  - [FromServices] - The request service injected as an action parameter

Status codes

Level 200
Success

200 - OK
201 - Created
204 - No Content

Level 400
Client Error

400 - Bad Request
401 - Unauthorized
403 - Forbidden
404 - Not Found
418 - I’m a teapot

Level 500
Server Error

500 - Internal Server Error
501 - Not Implemented
502 - Bad Gateway
503 - Service Unavailable

Formatters and content negotiation

- **Content negotiation** occurs when the client specifies an Accept header. The default format used by ASP.NET Core is JSON.
- **Accept header** is used to select the representation for a **response** based on a different media types like application/json, application/xml. This is set by the client and is considered an output formatter which tells the server how to serialize the request body.
- **Content-type header** is used by both client and server to identify the type of a resource in the request/response body. This is considered an input formatter which tells the server how to deserialize the request body.

Bye

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information



.NET Workshop

Course 8: .NET CRUD operations, services and DI

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Trimble

Agenda

- **CRUD** operations
- **C**reate resource
- **R**ead resource
- **U**pdate resource
- **D**elete resource
- Model validation
- Services
- Dependency injection (DI)
- Service lifetime

CRUD operations

- **CRUD** is an acronym that comes from the world of databases and it stands for **C**reate, **R**ead, **U**pdate and **D**elete.
- **CRUD** can be seen as a design paradigm which describes what operations can be performed on a set a data.
- Each letter can be mapped to a type of action that a user can perform on a set of data.

CRUD operations

- It’s possible to apply the **CRUD** concept to the world wide web and its underlying **HTTP** protocol.
- Considering that every URL points to a resource, that is a set of data you can access those with different HTTP verbs: GET, PUT, POST and DELETE.
- **CRUD** can be applied for API projects in order to describe how the resources can be manipulated and what are the operations supported by the API.

Create resource

- **Creating** a resource means that information about the newly created resource has to be provided to the server.
- **POST** request can be considered the create operation, the **C** in **CRUD**.
- A repeated **POST** request to the same path with the same data will create multiple resources of the same type but with a different identifier.

Create resource

- Related to a database, **creating** a resource means that the data is inserted in the database.
- Using a POST request, data is send to the server and leaves it up to the server to do something with it, so the URL of the final resource is not known before creation.
- The URL of the newly created resource can be specified using the location header in the response. This can be set using the method **CreatedAtRoute()** which will return a **201 Created** response and the location header set accordingly: more info here

Read resource

- **GET** is used for **reading** or retrieving data and it is the **R** in **CRUD**.
- This verb is idempotent so that making the same **GET** request twice returns the same response (unless the underlying data changed in the meantime).
- It's a safe operation because it doesn't alter the server-side state. When designing an API, **GET** should never be used for something that can modify a resource on the server.
- Related to a database, **reading** a resource means that a query to select all data from the database is done.

Model validation

- **Model validation** reports errors where data doesn't conform to any rules defined for the model.
- Model validation occurs before the execution of a controller action and after the model binding is done.
- At controller level, [ApiController] attribute uses as default a BadRequest response in case of a data conversion error.
- **Validation attributes** are used to specify validation rules for model properties and custom error messages displayed when a rule is not followed.

Model validation

Examples of built-in validation attributes:

- **[Required]**: Validates that the field is not null.
- **[Range]:** Validates that the property value falls within a specified range.
- **[RegularExpression]:** Validates that the property value matches a specified regular expression.
- **[StringLength]:** Validates that a string property value doesn't exceed a specified length limit.
- **[Url]:** Validates that the property has a URL format.

More validation attributes

Live code

Update resource

- **Updating** a resource means that information about an existing resource is changed.
- **PUT** request can be considered the update operation, the **U** in **CRUD**.
- **PUT** affects a resource identified by the URL based on the provided information for update. In case not all information has to change, this will remain with its default value.
- Doing the same PUT request twice will end with the same result.

Update resource

- Related to a database, **updating** a resource means that the existing data in the database is modified.
- As a response, usually 204 No Content is returned from an update request because the data is already known by the user of the API. This can be set using the **NoContent()** method.

Delete resource

- **DELETE** is the **D** in **CRUD** and removes one or more resources from the server.
- To delete a resource from server, **DELETE** request is used.
- Related to a database, **deleting** a resource means that the data is removed from the database.

  Live code

Trimble

.NET Services&DI

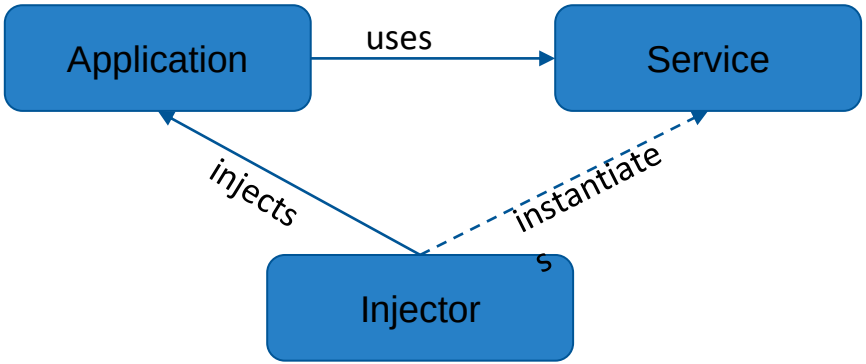
© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Service

- A **service** is a class which provides specific functionality for an application. Services are usually provided using Dependency Injection pattern. Two types of services:
- **Framework services**: these services are part of the .NET Core framework (IApplicationBuilder, IConfiguration, ILoggerFactory, etc.)
- **Application services**: these are the services that you create in your application; since the DI doesn't know them, you need to register them explicitly .

Dependency Injection

- **Dependency Injection (DI)** is a design pattern which allows the creation of dependent objects outside of the class that depends on them and provides those objects to a class through different ways.



Dependency Injection

- A **dependency** is an object that another object depends on. By using an instance of that dependency in your class and not using it through DI, you will end up with issues like:
  - Your class will depend directly on the class instantiated (the dependency). Creating a new instance in your class forces a tightly coupled connection (a hardcoded reference) of your class to the object instantiated.
  - Changing the implementation of the dependency class means to also do changes in your class.
  - Difficult to unit test your functionality.

Dependency Injection

- **Dependency injection** solves the problems previously described by:
  - The use of an interface or base class to abstract the dependency implementation.
  - **Registration** of the dependency in a **service container**.
  - **Injection of the service into the constructor** of the class where it's used. The framework takes on the responsibility of **creating** an instance of the dependency and **disposing** of it when it's no longer needed.

Dependency Injection - Register Services

- **Registration** of the dependency (service) is done in a **service container**. ASP.NET Core provides a built-in service container called **IServiceProvider**.
- Services are registered in **Builder.Services** in **Program** class

Service lifetime

- DI built-in container manages the lifetime of a registered service. The container will automatically create and dispose a service instance based on the specified lifetime.
- Services can be registered with one of the following lifetimes:
  - Transient
  - Scoped
  - Singleton

Service lifetime - Transient

- **Transient** lifetime services are created **each time they are requested** from the service container.
- Register transient services with **AddTransient** method and specify the type of the service.
- In apps that process requests, transient services are disposed at the end of the request.
- As a good practice, register your services as transient wherever possible. In this case, you don’t have issues with multi-threading and memory leaks and you know the service has a short life.

Service lifetime - Scoped

- **Scoped** lifetime indicates that services are created per scope. In a web application, each request creates a new separated service scope. That means scoped services are **created once per request** (that instance will be shared **in a single request**).
- Register scoped services with **AddScoped** method and specify the type of the service.
- In apps that process requests, scoped services are disposed at the end of the request.

Service lifetime - Singleton

- **Singleton** lifetime services are **created the first time they are requested**.
- **Singleton** lifetime services are created **only one time per application** and then used for the entire lifetime of the application.
- Register singleton services with **AddSingleton** method and specify the type of the service.
- In apps that process requests, singleton services are disposed when the ServiceProvider (built-in container service) is disposed on application shutdown.

Bye

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information



.NET Workshop

Course 9: MongoDB & SignalR

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Trimble

Agenda

- What is Mongo
- CRUD operation on a mongo database
- SignalR
- WebSockets

MongoDB

- **MongoDB** is a document-oriented **NoSQL database program** used for high volume data storage.
- **MongoDB** makes use of **collections** and **documents** and not tables and rows as in the relational databases.
- Documents consist of **key-value pairs** which are the basic unit of data in MongoDB. Collections contain sets of documents.
- Classified as a NoSQL database program, MongoDB uses **JSON** format for documents with optional schemas (shape and content of a document).

What is the best SQL? NoSQL

MongoDB

- The data stored in the **MongoDB** is in the format of BSON documents.
- BSON stands for Binary representation of JSON documents. In the backend, the MongoDB server converts the JSON data into a binary form that is known as BSON and this BSON is stored and queried more efficiently.
- The data model available within MongoDB allows you to represent hierarchical relationships, to store arrays, and other more complex structures more easily.

MongoDB - MongoDB.Driver

- In order to perform CRUD operations against the MongoDB database, members from **MongoDB.Driver** package are used:
  - **MongoClient** reads the server instance for performing database operations. The MongoDB database connection string is passed to the constructor of MongoClient class.
  - **IMongoDatabase** represents the Mongo database for performing operations.
  - **MongoCollection** object represents the collection for a specific database. CRUD methods are invoked on the collection.

MongoDB - CRUD operations

In order to perform **CRUD** operations on a collection methods from **IMongoCollection** are used:

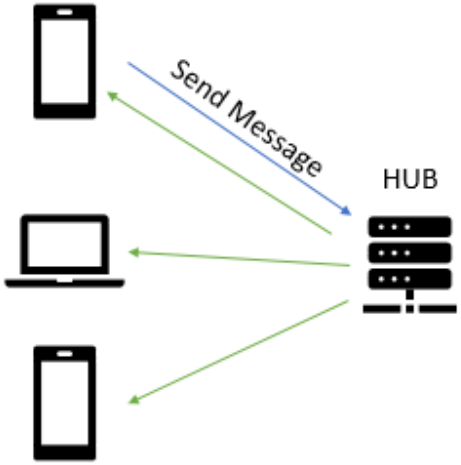
- Create: **InsertOne** or **InsertOneAsync** for inserting a single new document in the collection
- Read: **Find** or **FindAsync** for reading a document from the collection based on a filter

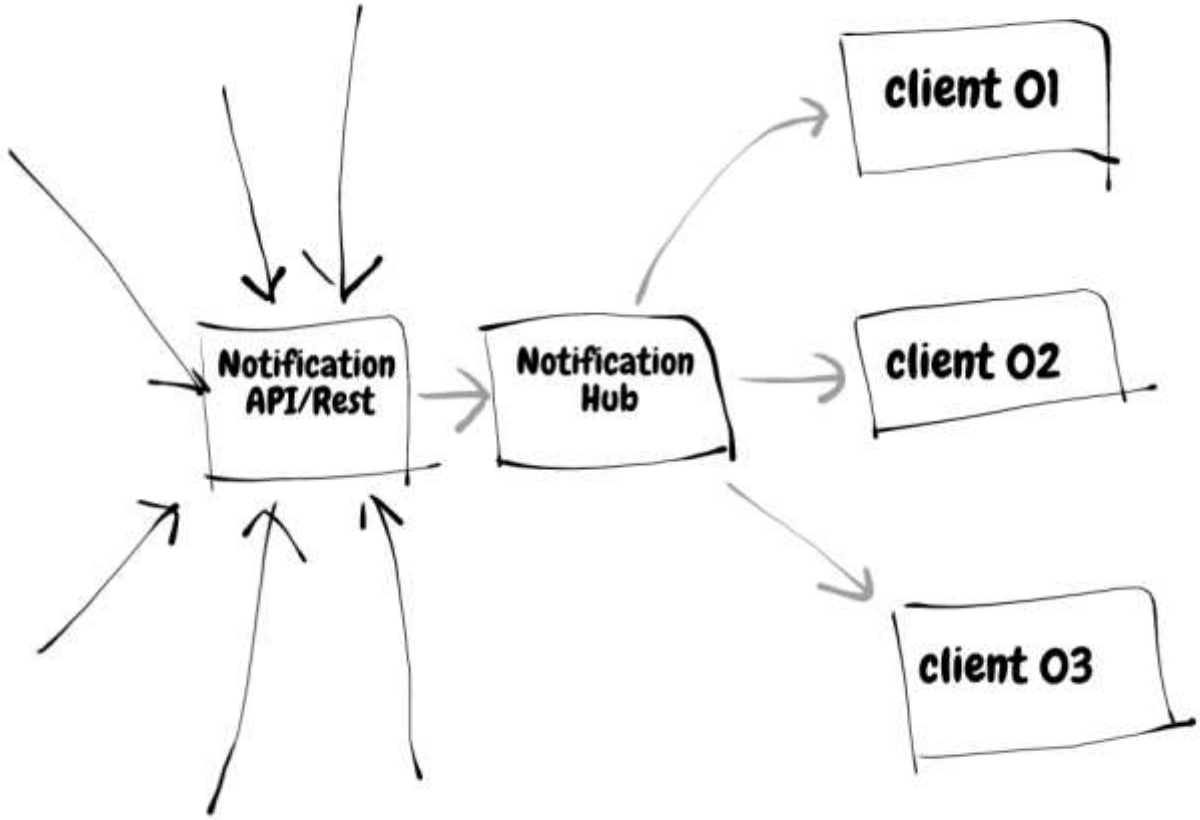
MongoDB - CRUD operations

- Update:
  - **UpdateOne** or **UpdateOneAsync** for updating a single existing document in the collection
  - **ReplaceOne** or **ReplaceOneAsync** for replacing a single existing document in the collection
- Delete: **DeleteOne** or **DeleteOneAsync** for deleting a single document in the collection

SignalR

- A free and open-source software library for Microsoft ASP.NET that allows server code to send asynchronous notifications to client-side web applications. The library includes server-side and client-side JavaScript components.
- SignalR takes advantage of several transports, automatically selecting the best available transport given the client's and server's capabilities. SignalR takes advantage of WebSocket, an HTML5 API that enables bi-directional communication between the browser and server. SignalR will use WebSockets under the covers when it's available, and gracefully fall back to other techniques and technologies when it isn't, while the application code remains the same




Notification API/Rest
Notification Hub
client 01
client 02
client 03

Example code

UI

```
this.connection.on('message_received', (body: any) => {
  this.notificationSubject.next(true);
});
```

API

```
await Clients.Caller.SendAsync("message_received");
```

```
this.connection.send("HasNewNotifications", userMail);
```

```
public async Task HasNewNotifications(string email)
{
    var notifications = (await _notificationService.GetNotificationsForUser(email));

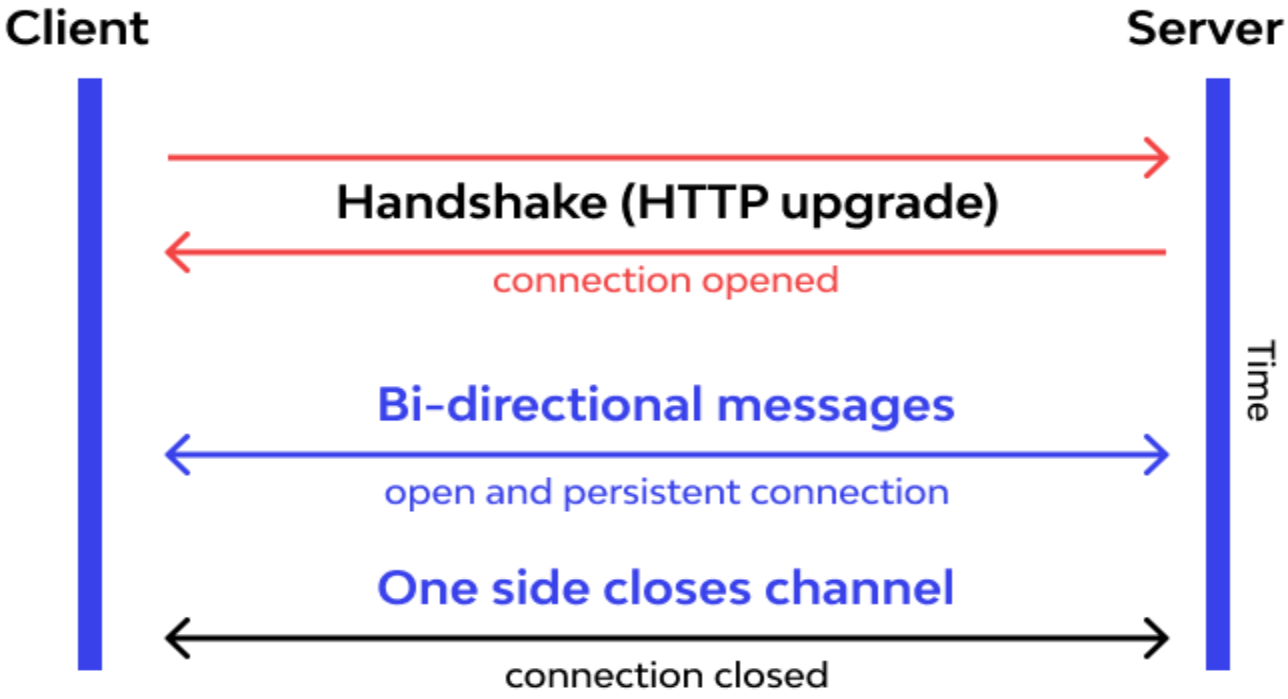
    if(notifications.Any())
    {
        await Clients.Caller.SendAsync("message_received", notifications);
    }
}
```

WebSockets

The WebSocket API is a technology that makes it possible to open a two-way interactive communication session between the user's browser and a server. With this API, you can send messages to a server and receive event-driven responses without having to poll the server for a reply.

https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

WebSocket


Client
Server
Handshake (HTTP upgrade)
connection opened
Bi-directional messages
open and persistent connection
One side closes channel
connection closed
Time

HTTP vs WebSocket

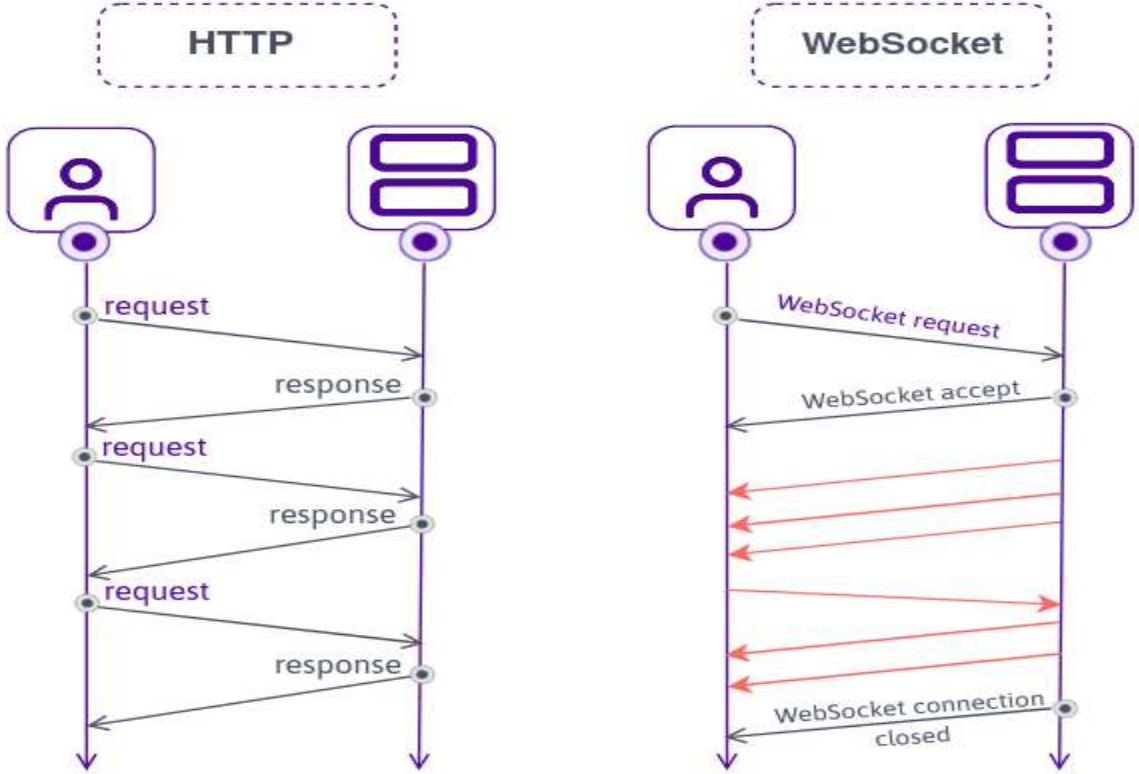
Http

- Unidirectional. For every request there is a response.
- Connection terminated after response.
- Requests use RESTful API.
- Best used for applications that don’t require 2-way connection.
- Responses can be cached.

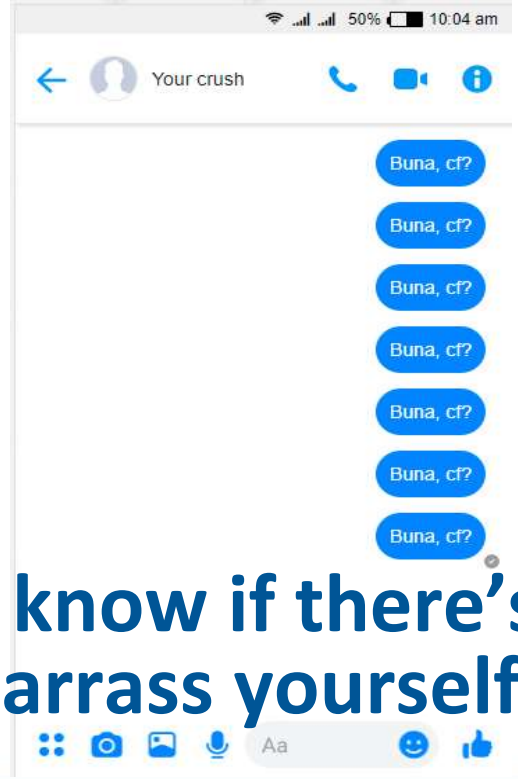
WebSocket

- Bidirectional.
- Connection is kept alive until terminated by client or server.
- Real time data.
- Best used for applications that require a stable connection.
- Responses cannot be cached.

HTTP vs WebSocket

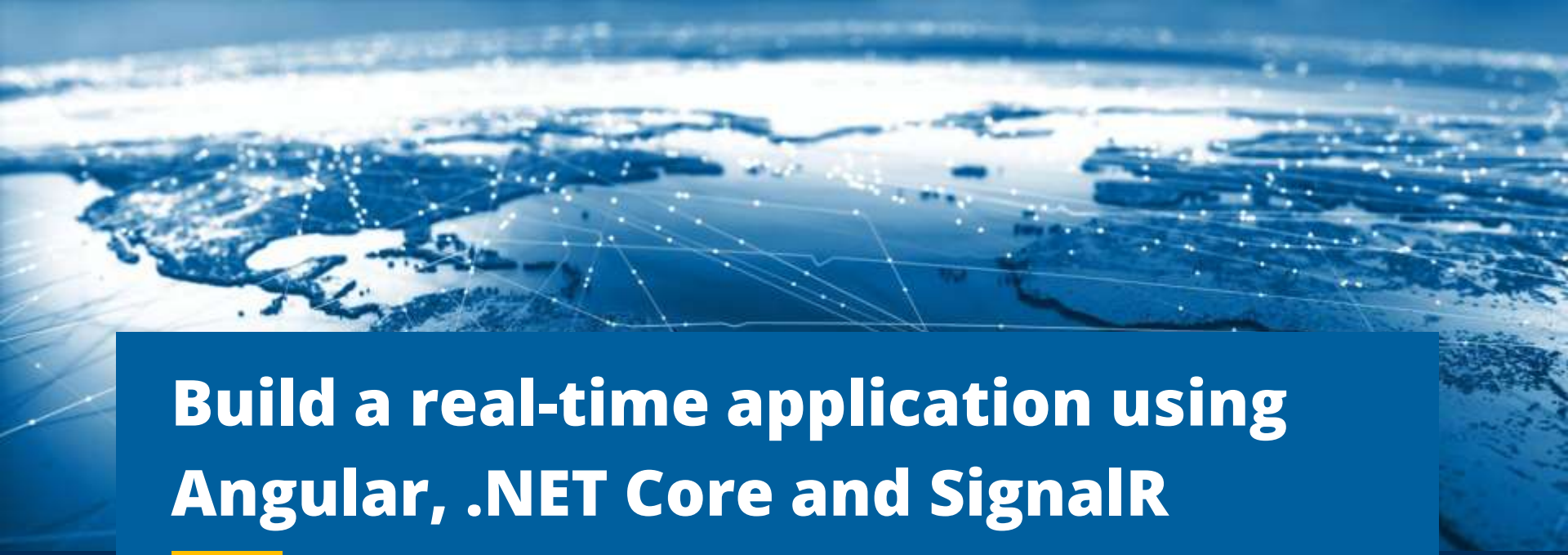


In WebSocket, the server will not always send a response. Just like your crush.



But at least you know if there's a connection before you embarrass yourself.

Bye!



Build a real-time application using Angular, .NET Core and SignalR

Course 10: Testing

© 2021 Trimble, Inc - All Rights Reserved - Confidential and Proprietary Information

Trimble

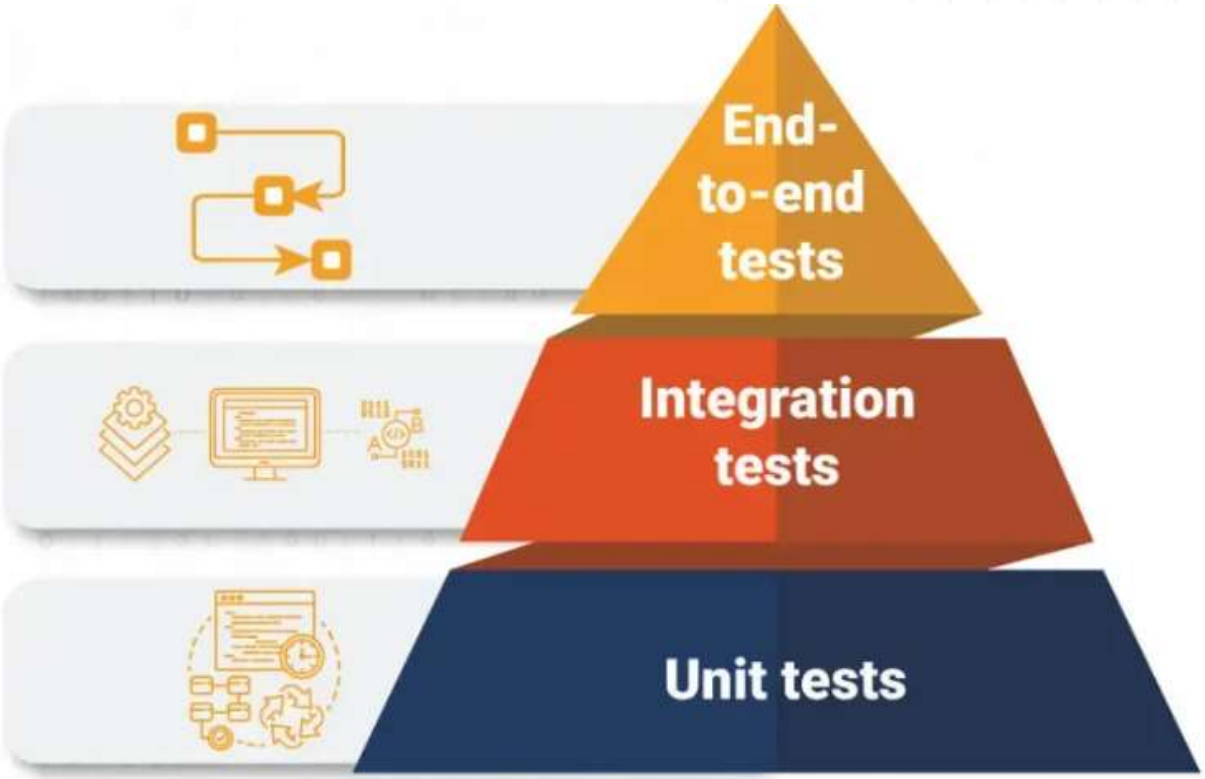
Agenda

- Automation testing
- Unit testing
- Unit tests C# NUnit
- Unit tests Angular Karma

Automation testing

- Automation testing uses an automation tool to execute the test case instead of a human manually executing the suite by following step-by-step instructions. So the main goal of automation testing services is to reduce the number of test cases that has to run manually instead of eliminating manual testing altogether.

Automation testing



Automation testing

- **Unit tests** have a limited scope and ensure that isolated code units function as expected. Unit tests should assess a single variable and not rely on external dependencies.
- **Integration tests** ensure that component interactions work as expected against external artifacts like databases. Assertions can test component API, UI, or the side effects of actions like database I/O, logging, etc.

Automation testing

- **End-to-end tests** should simulate real-world functionality. As this test checks assembled applications, it is also the most difficult to identify issues.
  The end-to-end tests are the most expensive to maintain and take long time

Unit tests

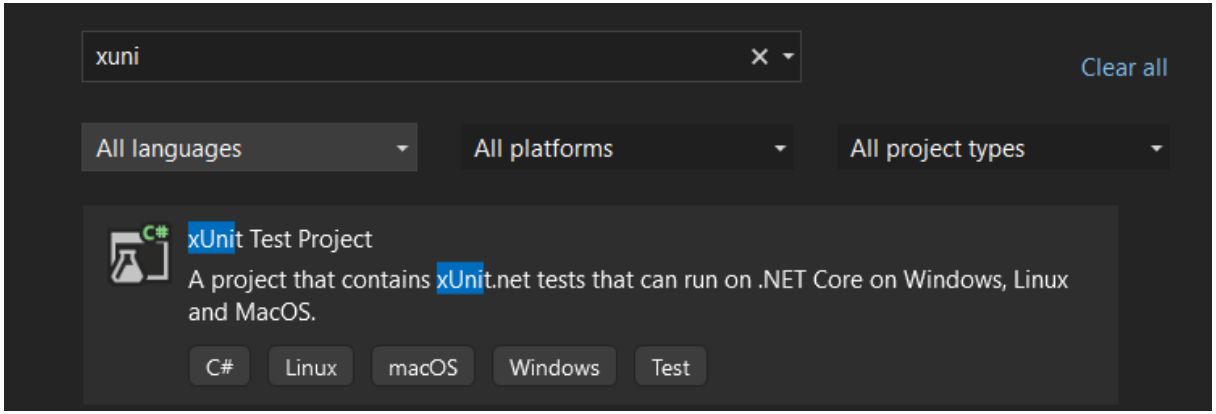
- A method where individual units of source code are tested.
- Concerned with functional correctness and completeness of individual programs units
- Are written by software developers to be sure that code behave as intended.
- Isolatable, repeatable, automatable, easy to write , easy to debug
- Example of unit tests: error handling, checking input values, correctness of output data, etc

Unit tests - Guidelines

- Keep unit tests small and fast
- Unit tests should be automated and non-interactive
- Make unit tests simple to run
- Fix failing tests immediately
- Keep testing at unit level
- Keep tests independent
- Name tests properly
- Keep tests close to the class being tested
- Design code with testing in mind
- Use explicit asserts

Unit tests for ASP.NET Core Web APIs

Unit tests are implemented based on test frameworks like xUnit.net, MSTest, or NUnit.We will use XUnit like a framework example.First time we should create a test project. In VS we should create a new XUnit test project



Unit tests for ASP.NET Core Web APIs

- **xUnit** is a free and open-source Unit testing framework for .NET development
- **xUnit** has many attributes :
  - **[Fact]** :is used by xUnit in .NET which identifies the method for unit test
  - **[Theory] :**is used when you want send some parameters to the test. It is similar to Fact in a way that test runner will execute this method but it also specifies that we will be sending parameters to the test data.
  - **[InlineData]:** is a way to provide parameters to the test method. This is used along with Theory attribute.

Unit tests for ASP.NET Core Web APIs

When writing unit tests it is usually the practice to follow the **AAA** principle (**Arrange**, **Act**, and **Assert**):

- **Arrange**: this is where you would typically prepare everything for the test.
- **Act**: this is where the method we are testing is executed
- **Assert**: this is the final part of the test where we compare what we expect to happen with the actual result of the test method execution

Unit tests for service - Get method

```
/// <summary>
/// GET Method summary
/// </summary>
/// <returns>OK</returns>
[HttpGet]
0 references
public async Task<IActionResult> Get()
{
    var announcements = await _announcementCollectionService.GetAll();

    if (announcements == null || !announcements.Any())
    {
        return NotFound();
    }

    return Ok(announcements);
}
```

Unit tests for controller - Get method

- We should mock the AnnouncementCollectionService call if not will go on default implementation, which communicate with database
- We will have 3 cases tests:
  - when the announcement list is null
  - When the announcement list is empty
  - when the announcement list contains data
- In all cases we should test what IActionResult is returned and what values has.

  Live code

Unit tests for controller - Get method

How we can mock the data from service:

- Create service class and implement interface

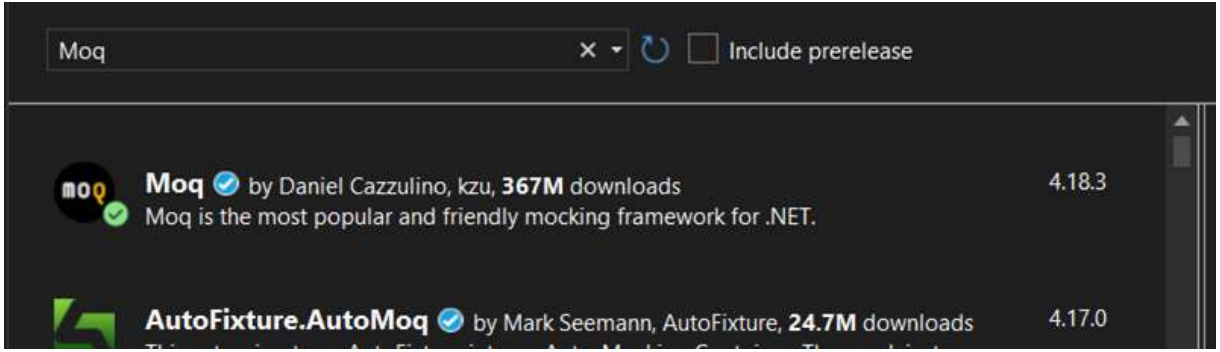
  Live code

- Using a package for mocking data:
  - **Moq:** https://www.nuget.org/packages/Moq

  Live code

Moq package - install

- Select the project you want to add the reference to Moq.
- Open Nuget for this project.
- Select "Browse" than type "moq" at the search box.
- Select "Moq" and than click on Install.



Unit tests in Angular

Unit testing in Angular refers to the process of testing individual units of code.An Angular unit test aims to uncover issues such as incorrect logic, misbehaving functions, etc. by isolating pieces of code.

The test script which ends with **.spec.ts**, is always added when create a component, pipe, service, etc.



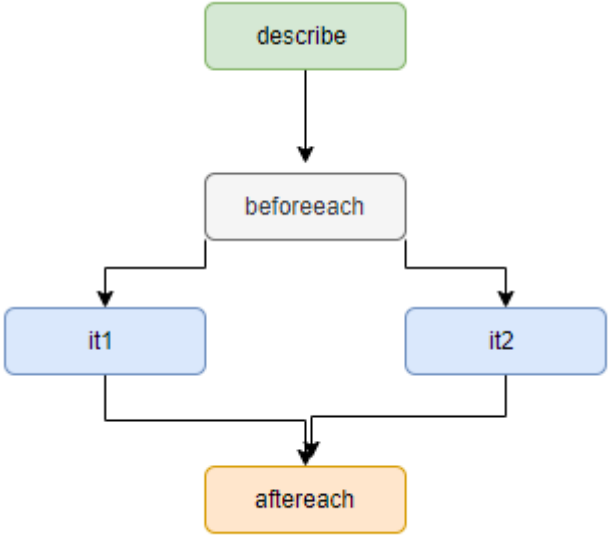
Unit tests in Angular - runner



Karma is a JavaScript test runner that runs the unit test snippet in Angular. By default, Angular runs on Karma. In an angular project **karma.conf** contains the configuration for Karma runner.

Jasmine is a testing framework and it is used to write unit tests in Angular

Unit testing Angular



Unit testing Angular

- **describe** container contains different blocks (it, beforeEach, xit, etc.).
- **beforeEach** runs before any other block. Other blocks do not depend on each other to run. Here you should create component to test with mocked service, etc.
- **afterEach** runs after any other block.
- **it** contains the test content

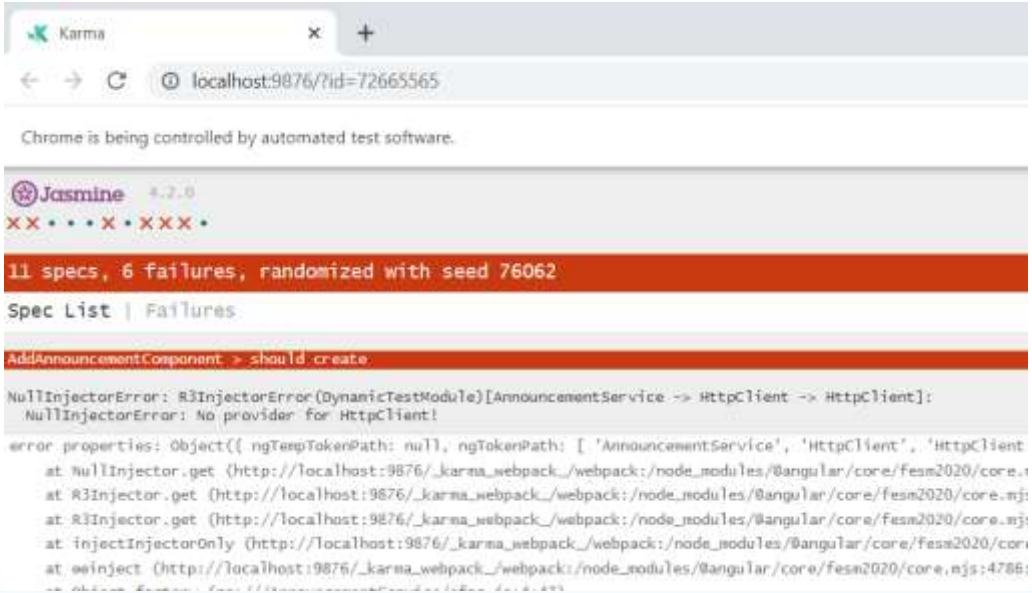
Live code

Unit testing Angular - mocking

- **Mocking with fake classes:**
  - we can create a fake service and provide it in testing module instead of the service
    Live code
- **Mocking with Spies**
  - We can create a real instance of service and inject it in component and create a spy on our service method
    The basic syntax of using a Spy is:
    `spyOn(Object to spy, function or property to spy)`
    Live code

Unit testing Angular - run tests

To run unit tests on Angular you should use **ng test** command .
The ng test command builds the application in watch mode, and launches the Karma test runner. **LIVE CODE**



Bye!